Computer Networks

计算机网络（CST2398）

2025-2026学年第二学期（26-Spring）

计算机科学与技术学院 王洁

wangjie_tongji@tongji.edu.cn

同济大学 TONGJI UNIVERSITY

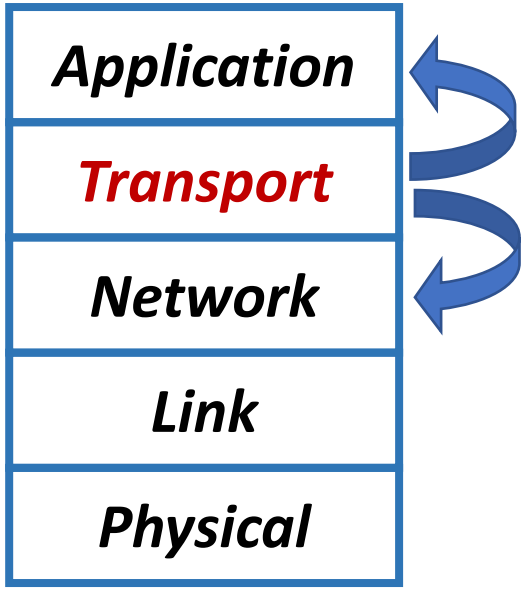
06 Transport Layer

Contents of this Chapter

- ***Why: Is Network+Datalink+PHY enough? Why an extra layer?***
  - *Transport Layer Functionalities*
  - *Services provided to the upper layer (Application)*
- ***What: TCP***
  - *Message(**Segment**) Format – headers*
  - *Connections and Endpoints*
- ***How: to provide reliable data transport between application processes?***
  - *Connection Establishment/Termination – 3-way/4-way handshakes*
  - *Flow Control and Congestion Control – "fancier" sliding window*

Transport Layer: at the heart of the hierarchy

- ***Process-to-process Data Transport***

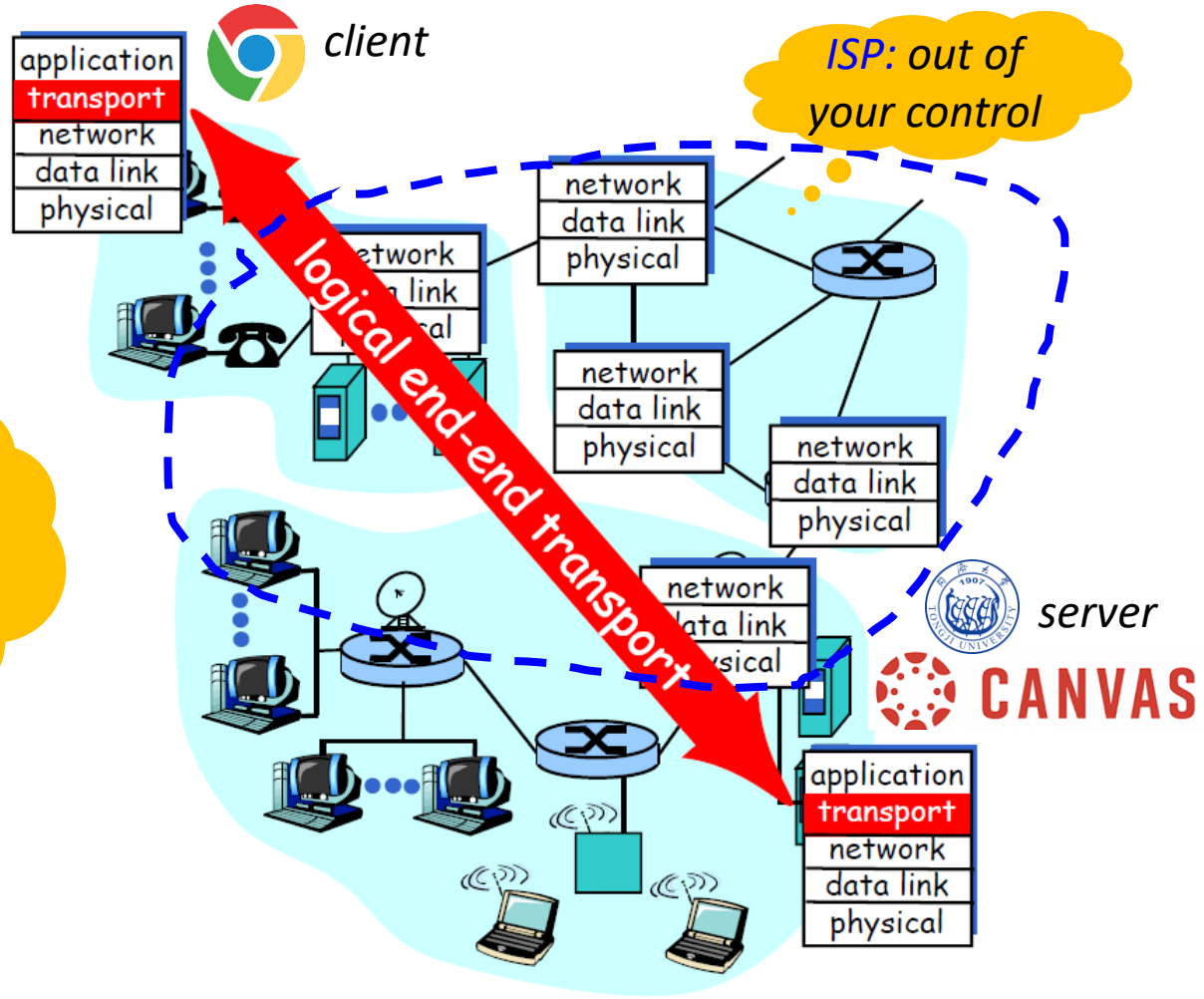


Providing services to

Using services from

Shield Application from networking details

***Transport**: provides logical communication between application processes running on different hosts.*



Services Provided to the Application Layer

- ***Two types of services***
  - ***Connectionless*: User Datagram Protocol (UDP)**
    - *Unreliable, unordered, unicast/multicast delivery – "best-effort", but quick and easy!*
  - ***Connection-oriented*: Transmission Control Protocol (*TCP*)**
    - *Reliable, in-order, unicast delivery*
      - *Connection setup*
      - *Flow control*
      - *Congestion control*

BE on BE - easy

Reliable on BE –How?

Services provided by the Network layer

- ***Connectionless Services – IP***
- *Connection-oriented Services*

- ***Implementation:***
  - *Mostly adopted: in OS kernel, or as a library package in network applications*

Design Goals of Transport-TCP

- ***Reliability***
  - *No error, guaranteed in-order data delivery*
- ***Flow Control***
  - *Do not swamp a slow receiver.*
- ***Congestion Control***
  - *Do not swamp a slow network.*

Error & Flow Control, sounds familiar?

Sounds familiar?

Aspects	*DLC*	*Transport*
Communication over	*a physical channel*	*a network*
Explicit Addressing	*not always required*	*required*
To establish a connection is	*simple (single connection)*	*difficult (multiple, variable connections)*
Order of packets is	*preserved over channel*	*not preserved*

Aspects	*Network*	*Transport*
Run on	*Every device (mainly routers)*	*End devices*
Devices owned by	*ISP*	*You*

- ***Conclusion: Transport is needed by the application to shield from networking details.***

User Datagram Protocol (UDP)

- ***Connectionless, unreliable service***
  - *Upper Layer (Application) needs to deal with*
    - *Packet loss*
    - *Duplication*
    - *Out of order delivery…*

Best Effort, like IP

- ***Main Functionality added: port abstraction***
  - ***Why****: to allow multiple communication endpoints on the same host*
  - ***Addressing****: IP + Port #*

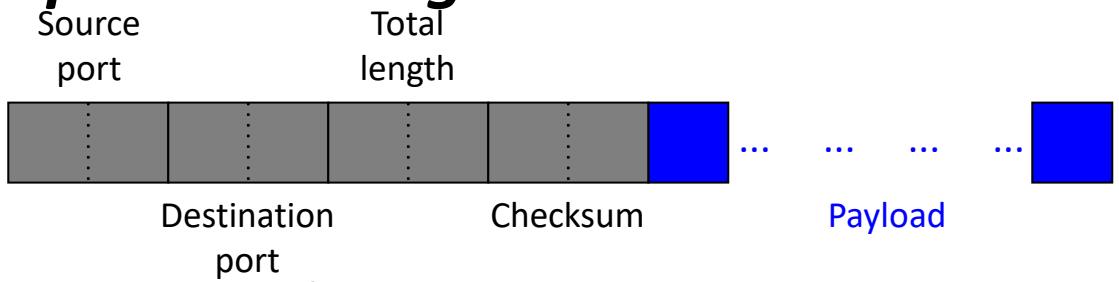
Multiplexing

Same as TCP

- ***User datagram encapsulated within IP datagram***

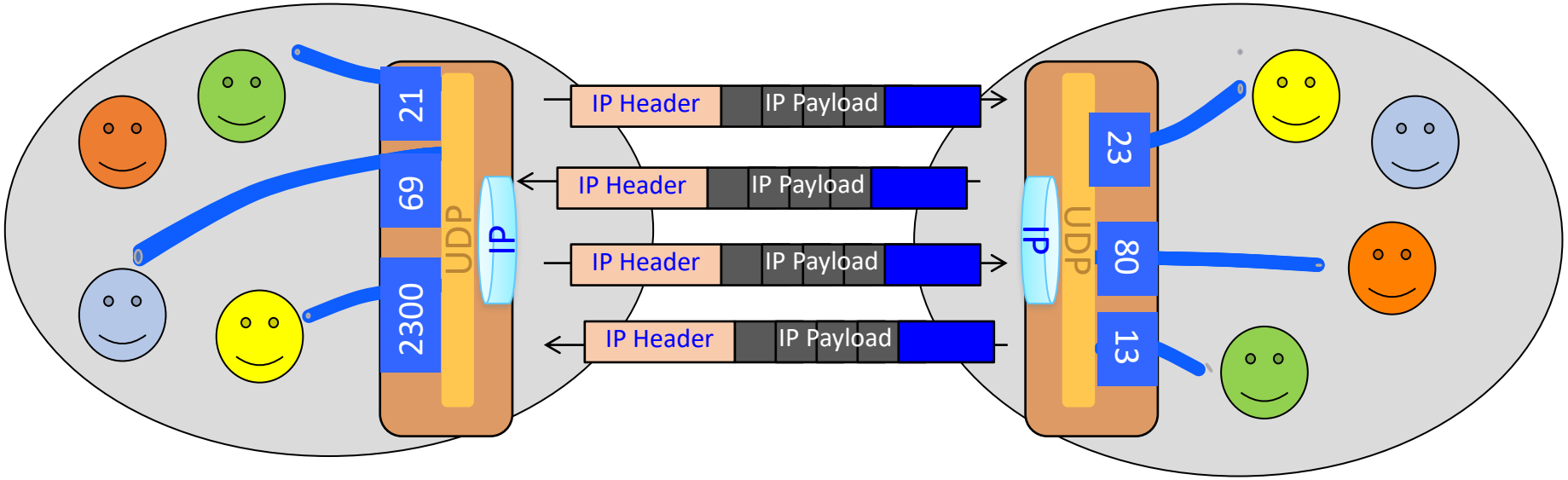
UDP Datagram Format

- ***Ports: abstraction of endpoints, a positive integer***
  - *Source Port: 2 Octets/Bytes*
  - *Destination Port: 2 Octets*



  - *OS provides an interface for process to specify/access ports*
  - *Synchronous access to ports (most cases)*
    - *Process blocks a port when it is unavailable*
    - *Buffered (using a finite queue)*
- ***Length (Header + data): 2 Octets***
- ***Checksum: 2 Octets***

UDP Multiplexing and Demultiplexing



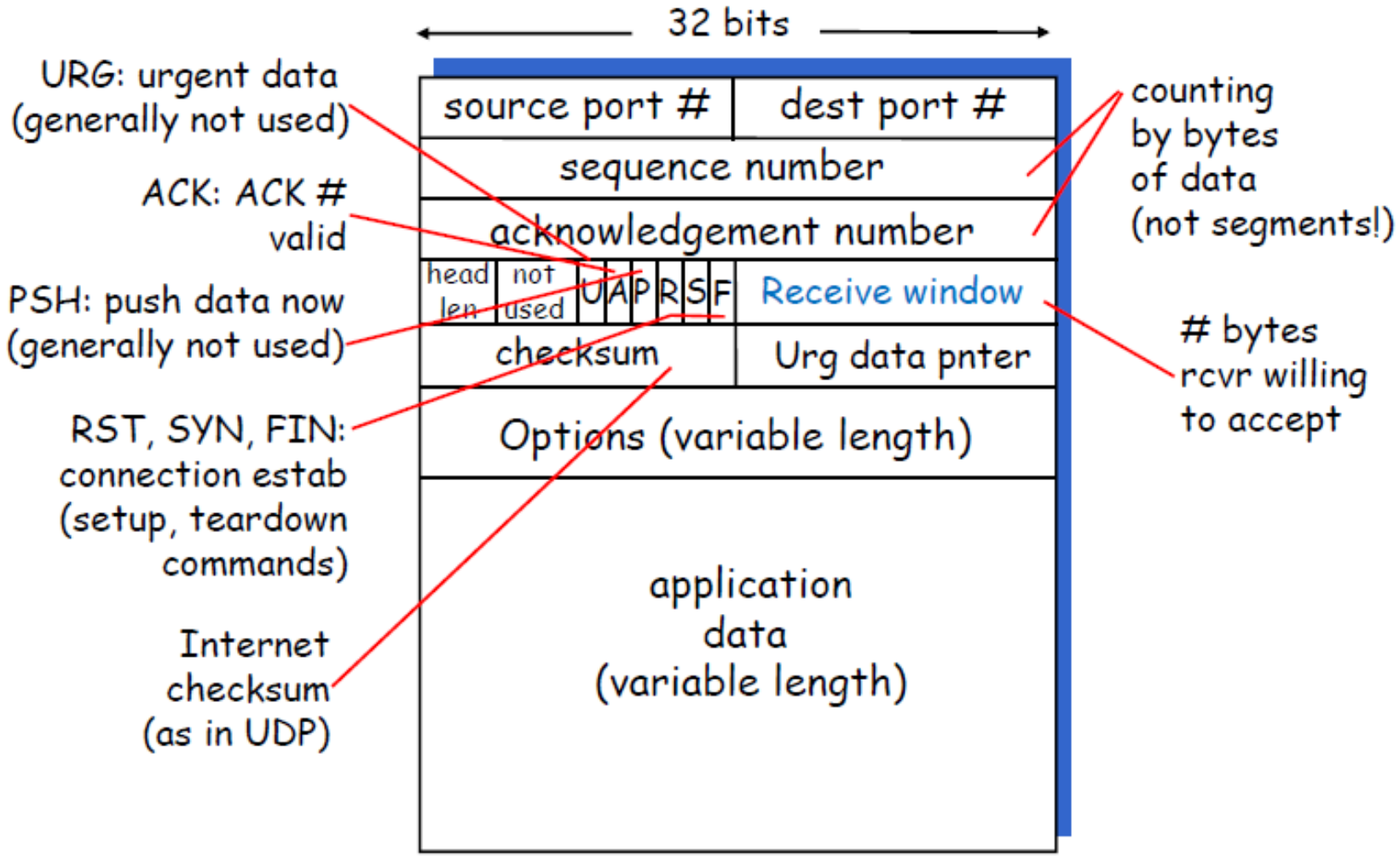
- ***Port number selects process***
  - *< 1024: well-known ports, require processes to have super-user access to bind*
  - *1024 and above: temporary ports, some platform further divides "registered" (< 49152) and "dynamic" ports*

6.1
TCP Overview

Basics of TCP

- ***TCP: Transmission Control Protocol*** *(RFCs 793, 1122, 1323…)*
- ***Data unit: Segment*** *(usually contained in a single IP datagram)*
- ***Reliability achieved with:***
  - *Acknowledgments (so Seq. #s are needed)*
  - *Timeout and retransmissions*
  - *Checksums on header and data*
- ***Flow and Congestion Control: Sliding Window***
- ***Note:*** *several versions*

TCP Segment Structure



- *Src./Dst. Port #*
- *Seq. #*
- *ACK #*

Stream of bytes

- *Recv. Window size*
- *Flags:*

Flow Control

  - *ACK*
  - *RST*
  - *SYN*
  - *FIN*

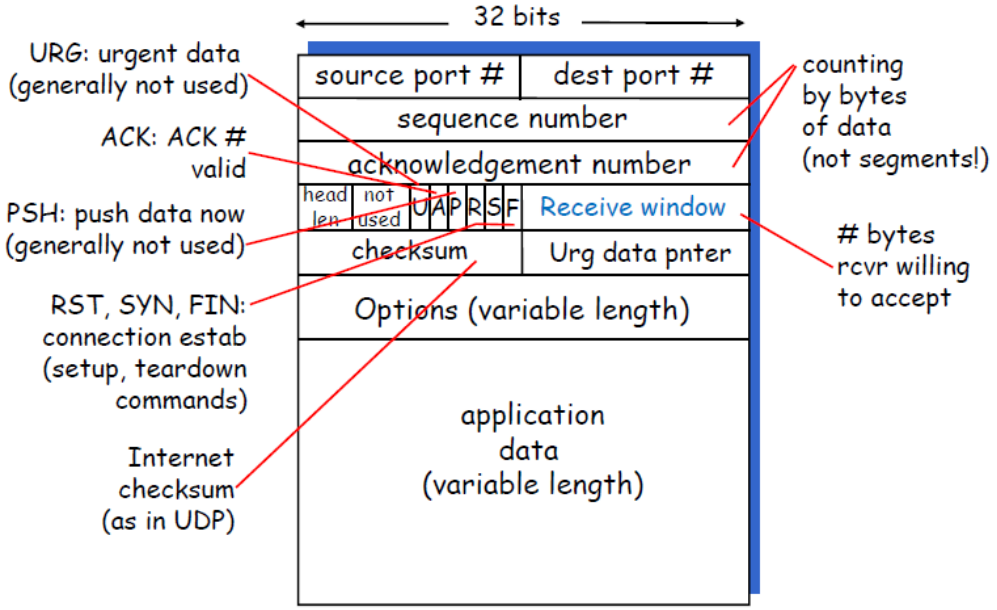
Connection

TCP Header Fields (1/3)

- ***Sequence number (Seq. #)***
  - *Every byte in the data stream is numbered.*
  - *Seq. # = number (in the sender's byte stream) of the first data byte in this segment*
  - *Wraps around after 2^{32} -1*
- ***ACK number (ACK #)***
  - *The next Seq. # the sender of the acknowledgment expects to receive*
  - *Seq. # + 1 of last successfully received consecutive data byte (Classic TCP)*
  - *Valid field only when ACK flag = 1*
- ***Header length***
  - *Length of header (in bytes) / 4*
  - *With the max. value of 15 (2^4-1), i.e., header cannot exceed 60 bytes*

TCP Header Fields (2/3)

- ***Receiver Window size***
  - *Number of bytes (starting with the one specified in the ACK field) that receiver is willing to accept*
  - *16 bits long; max value is 65535*
  - *Used for flow control*
- ***Checksum***
  - *Also used in UDP*
- ***Urgent pointer***
  - *Seq. # of the last byte of urgent data (later)*
  - *Added to the Seq. # of segment*



- ***Options***
  - *The Most common option is maximum segment size (MSS, code 2)*
    - *MSS must ≤ interface MTU – 40 bytes. Default value : 536*

TCP Header Fields (3/3)

- ***Flags in the Control Field***

Flag	Description
URG	*The value of the urgent pointer is valid*
ACK	*The value of the ACK # is valid*
PSH	*Push the data (pass data to receiver as quickly as possible)*
RST	*The connection must be reset*
SYN	*Synchronize the Seq. # during connection establishment*
FIN	*The sender has no more data to transmit.*

TCP Connections and Endpoints

- ***Connection:*** *a pair of* ***endpoints***
  - *The fundamental abstraction of TCP*

- ***Client*** *– the initiating host*
- ***Server*** *– the receiving host*

- ***Endpoint:*** *(host_IP_address, port #)*
  - *Port # is also used by UDP.*
    - *Port # in UDP -> single object/queue*
    - *Port # in TCP can by shared by multiple connections on the same host.*
  - *Port # ranges from 0 to 65535.*
    - *Well-known ports: 0 to 1024 (Read RFC 1700 for more information)*

    e.g. 20/21-FTP, 22-SSH, 25-SMTP, 53-DNS, 80-HTTP, 156-SQL Services…

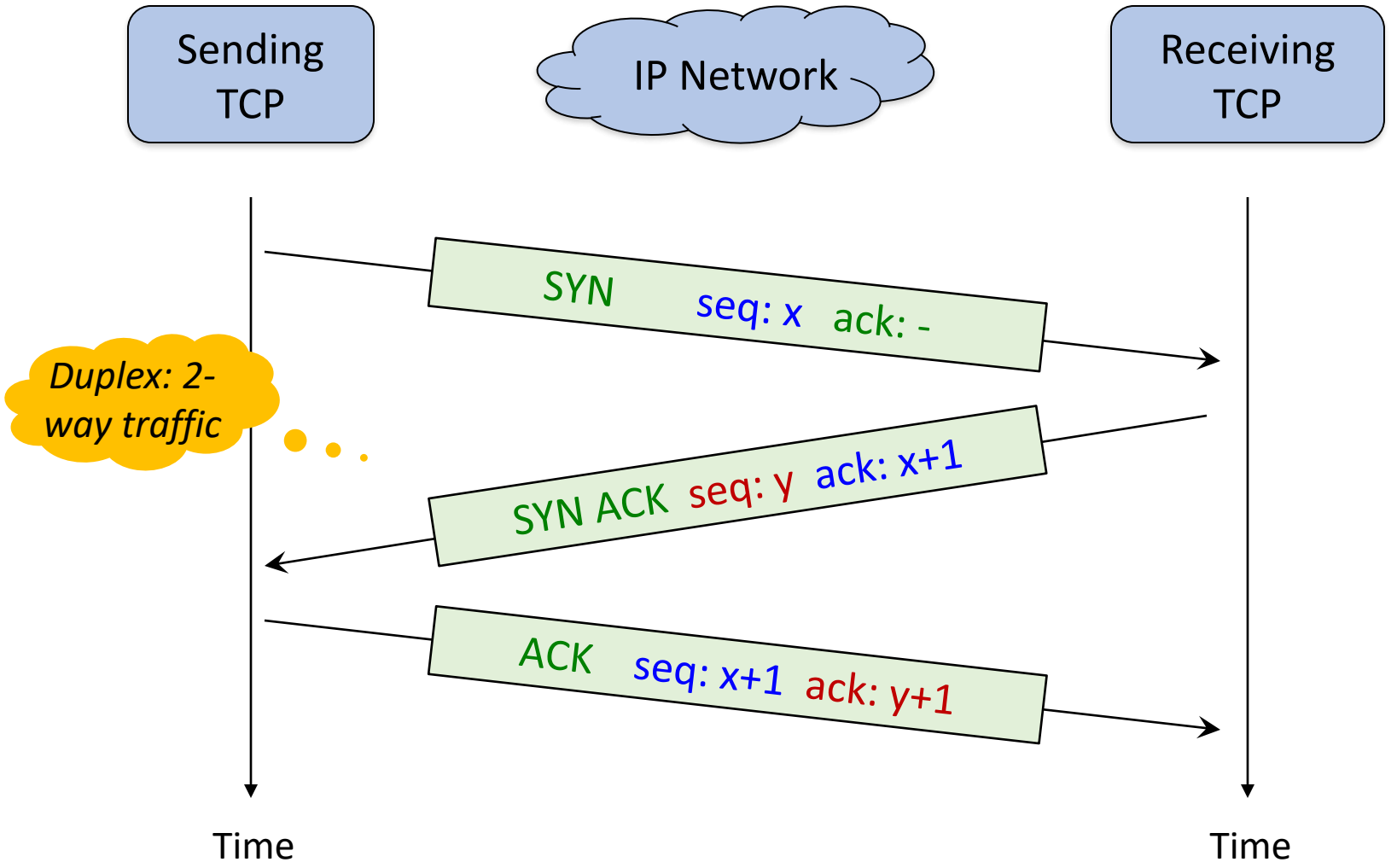
Design issues in Connection Establishment

- ***Will this simple approach (2-way handshake) work?***
  - *Sender: Connection Request*
  - *Receiver: Connection Accept*
  - *Seq. # for a new connection always starts at 0.*
- ***Nope. Problem? Delayed duplicates (of requests and accepts)***

Further, why?

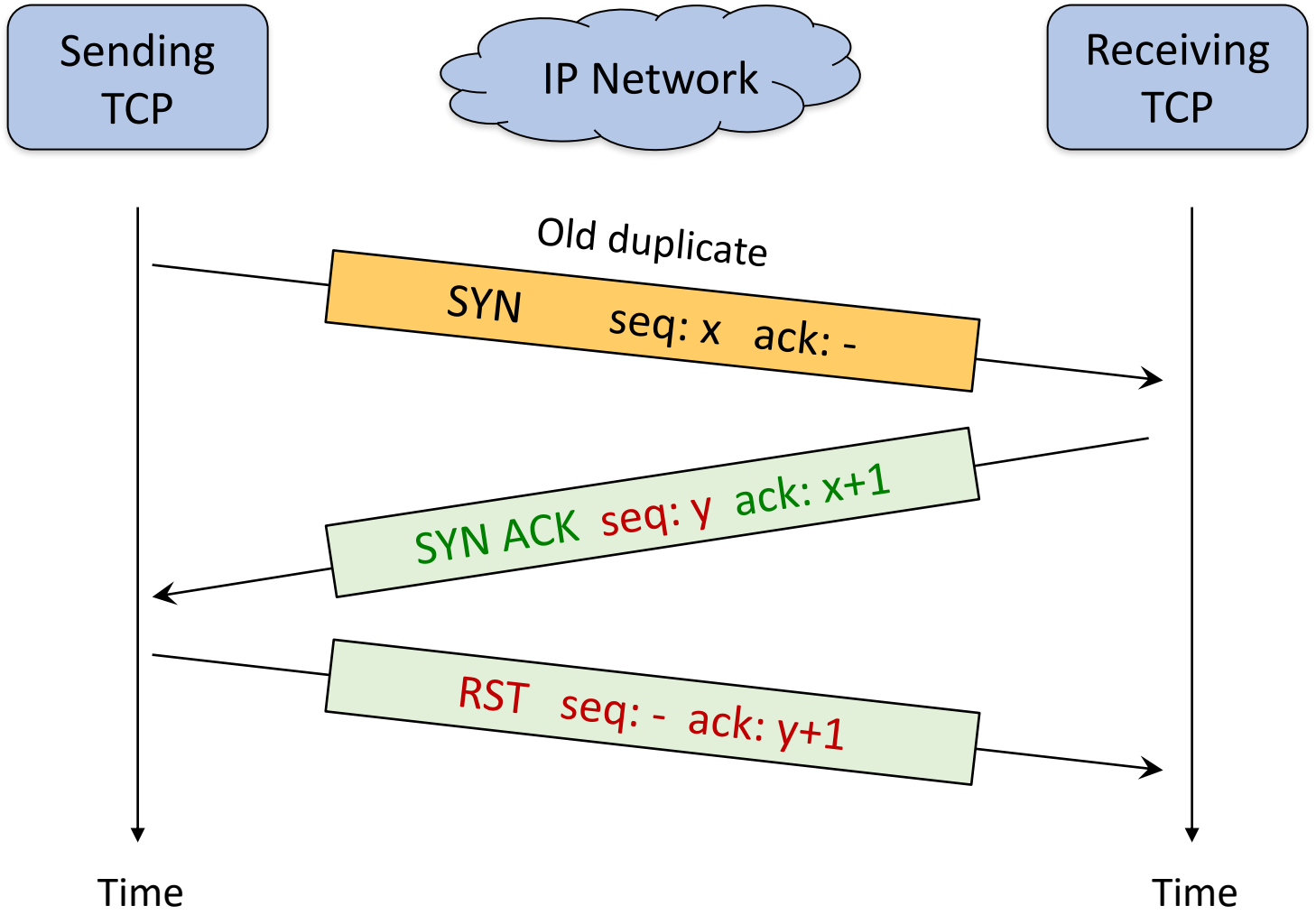
- ***Solution(s):***
  - *Limit packet lifetime T = max segment lifetime (MSL)*
  - *Long Seq. #: wrap around time > 2T (so that a packet and its ACK both die.)*
  - *Use a different **Initial Seq. # (ISN)** with each connection request.*
  - *Ignore additional requests for the same connection after its establishment.*
  - *Wait for 2T after a host crashes (so that old packets all die off.)*

Connection Establishment: 3-way Handshake

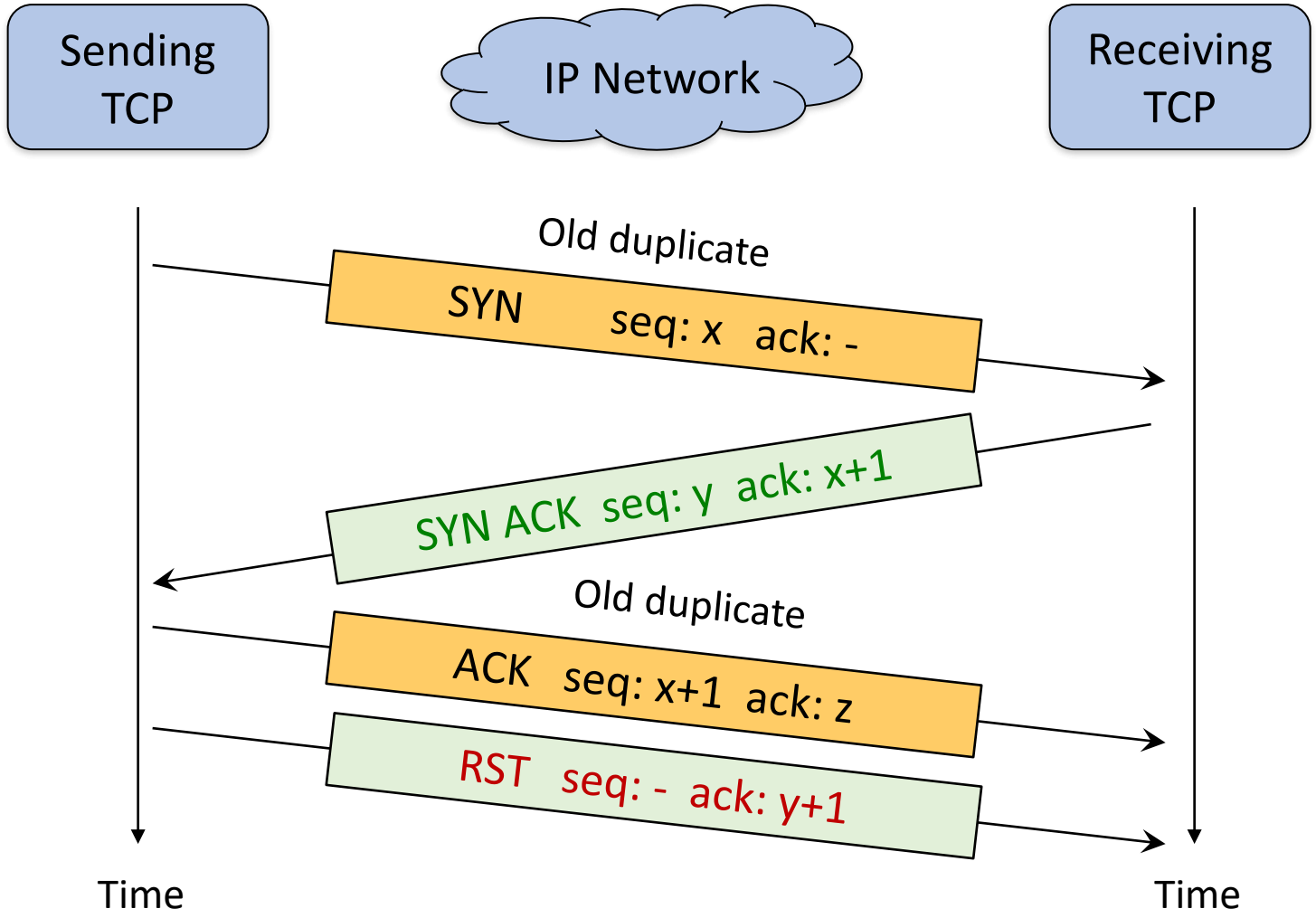


- ***Normal Flow***
  - *3-Way Handshake*
    - *SYN*
    - *SYN ACK*
    - *ACK*
  - *Corresponding relationship in*
    - *Seq #*
    - *ACK #*

3-Way Handshake – Duplicate SYN

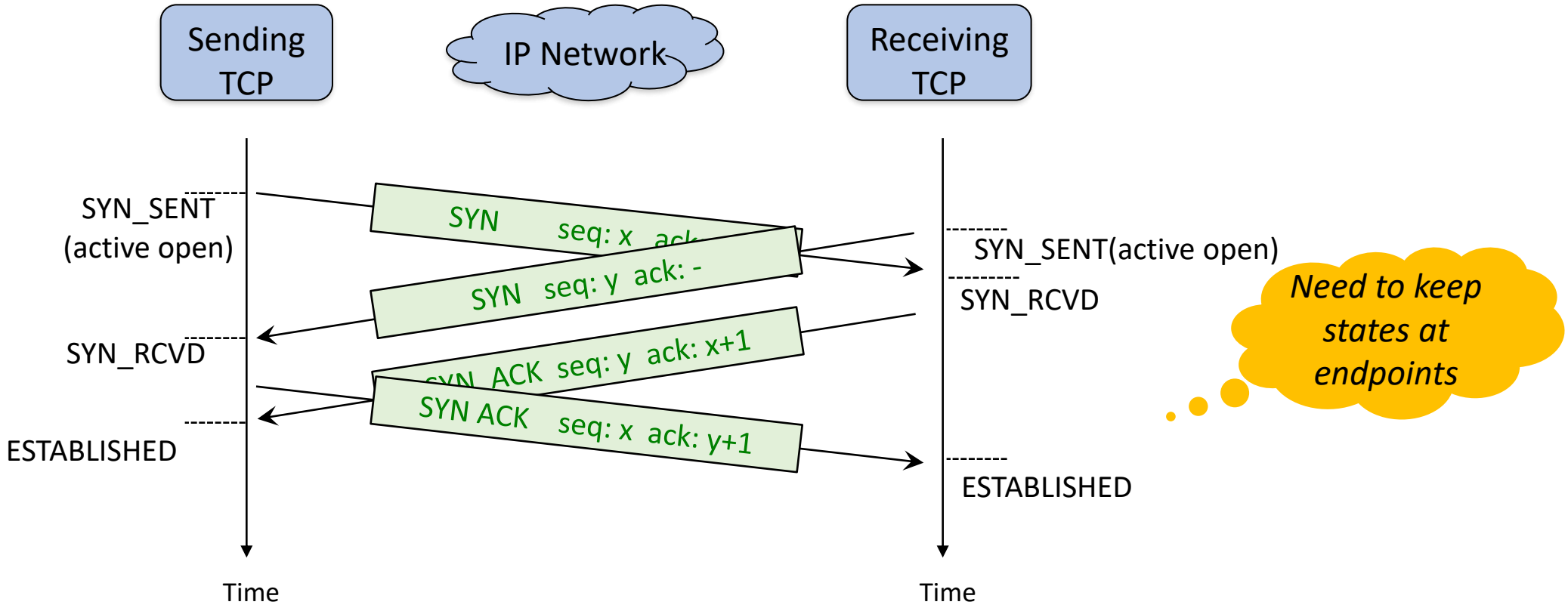


3-Way Handshake – Duplicate ACK



3-Way Handshake – Simultaneous Open

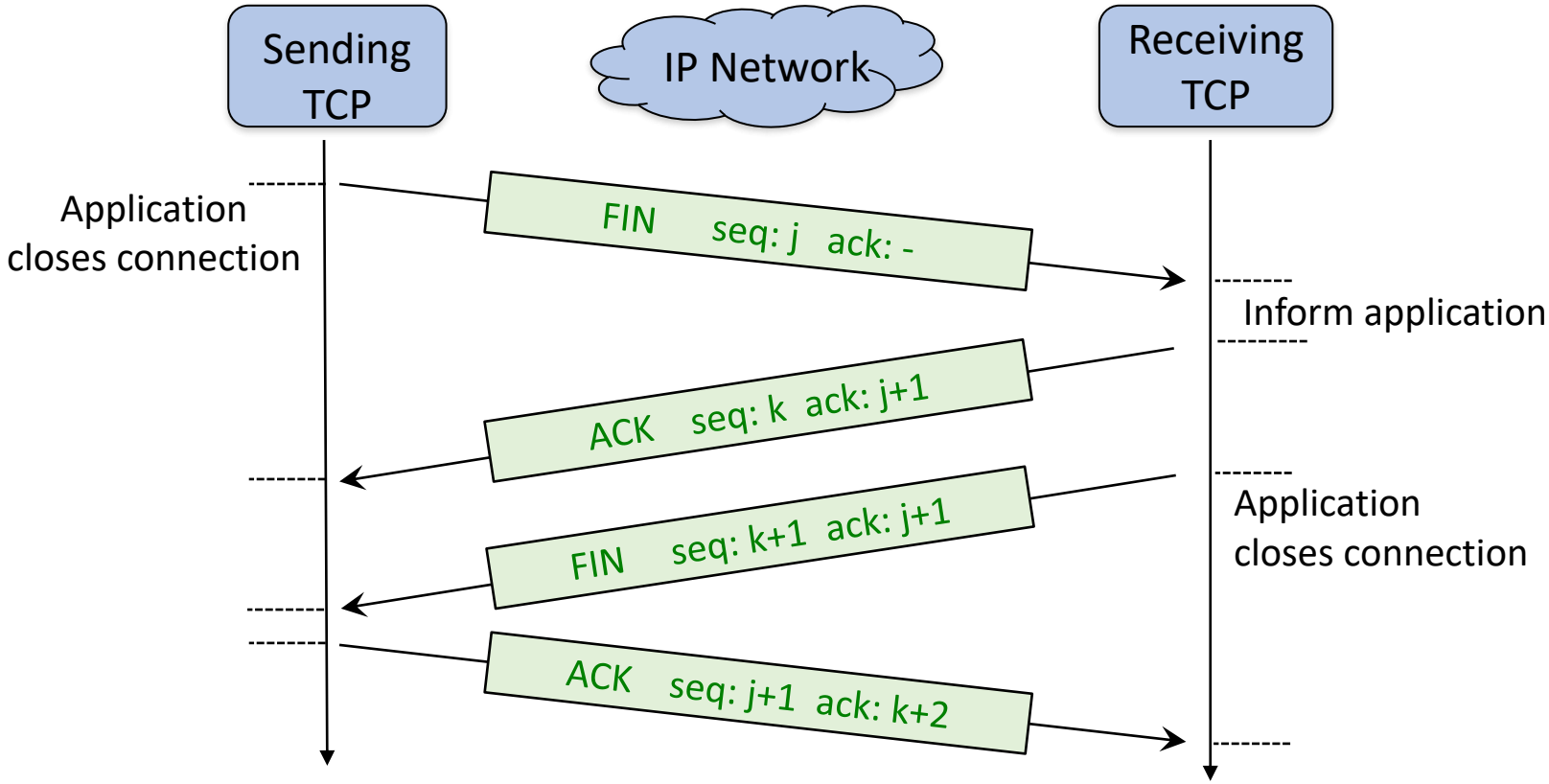
- ***A connects with B, B connects with A at the same time***
  - *Only one connection will be established! Using only 2 ports (one on A, one on B)*



Connection Termination – 4-way Handshake

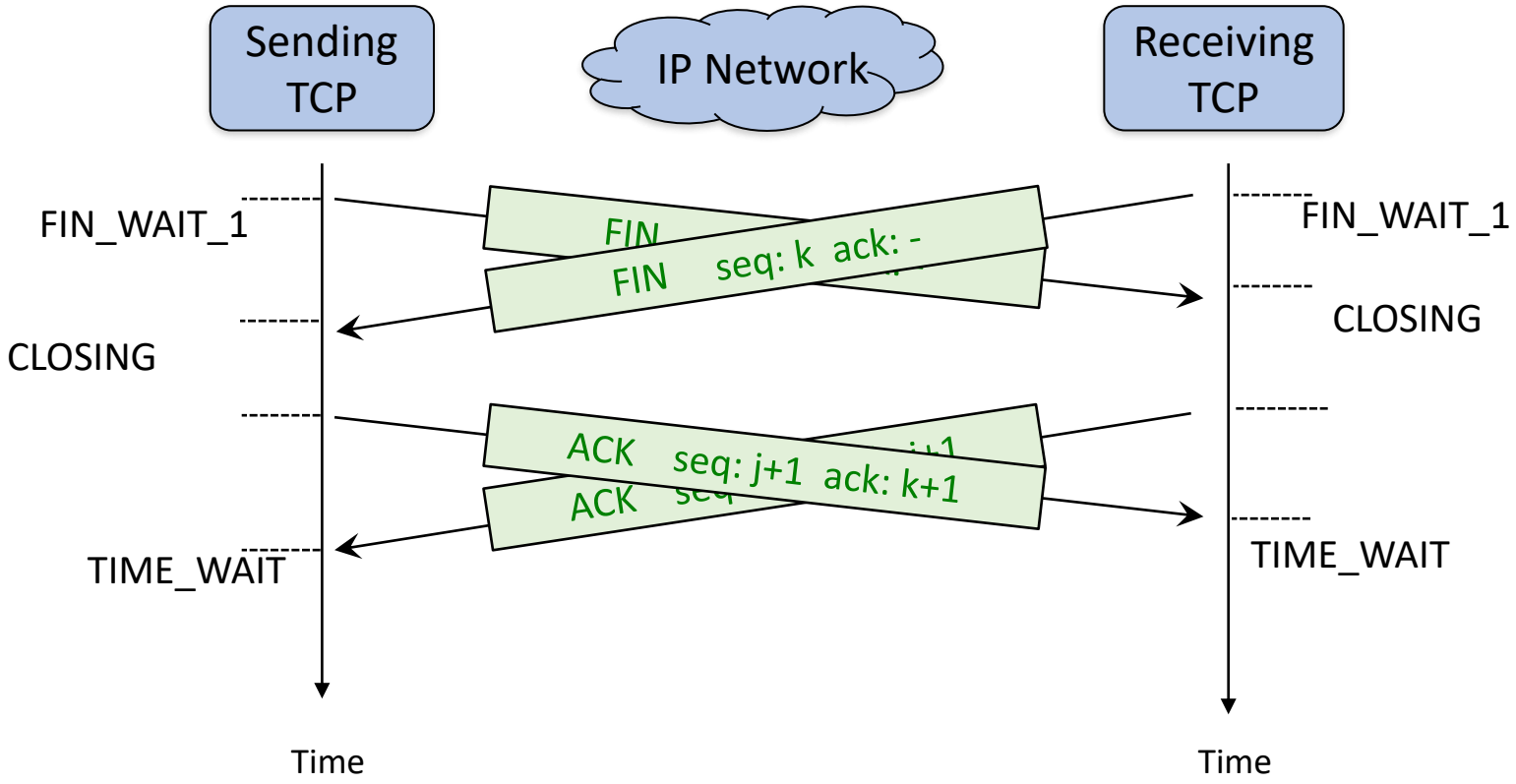
- ***4-way Handshake: 2-way in each direction***
  - *After closing connection in one direction, the other direction can still work.*

Half-close



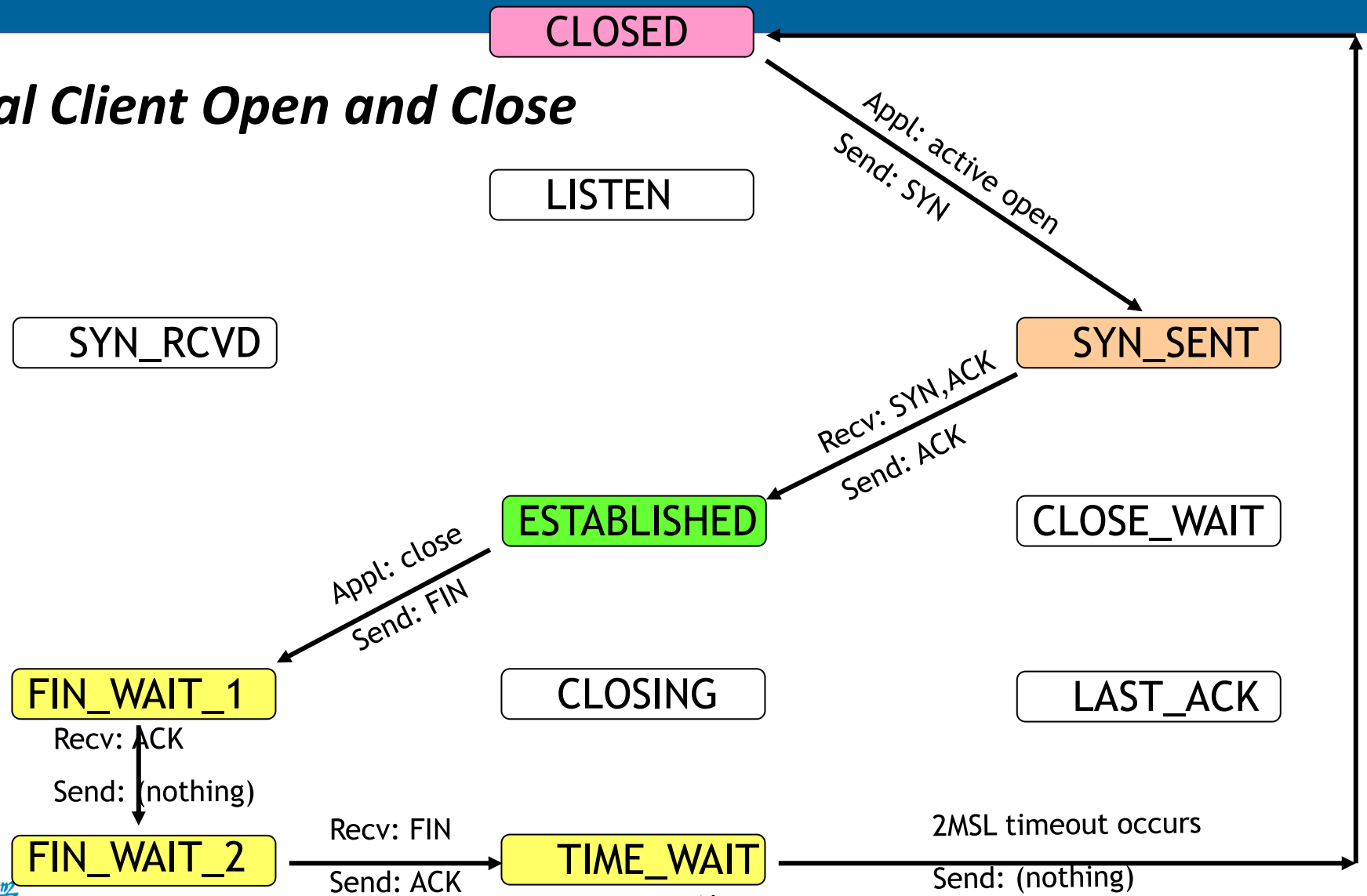
Simultaneous Close

- ***A sends FIN to B, B sends FIN to A at the same time.***



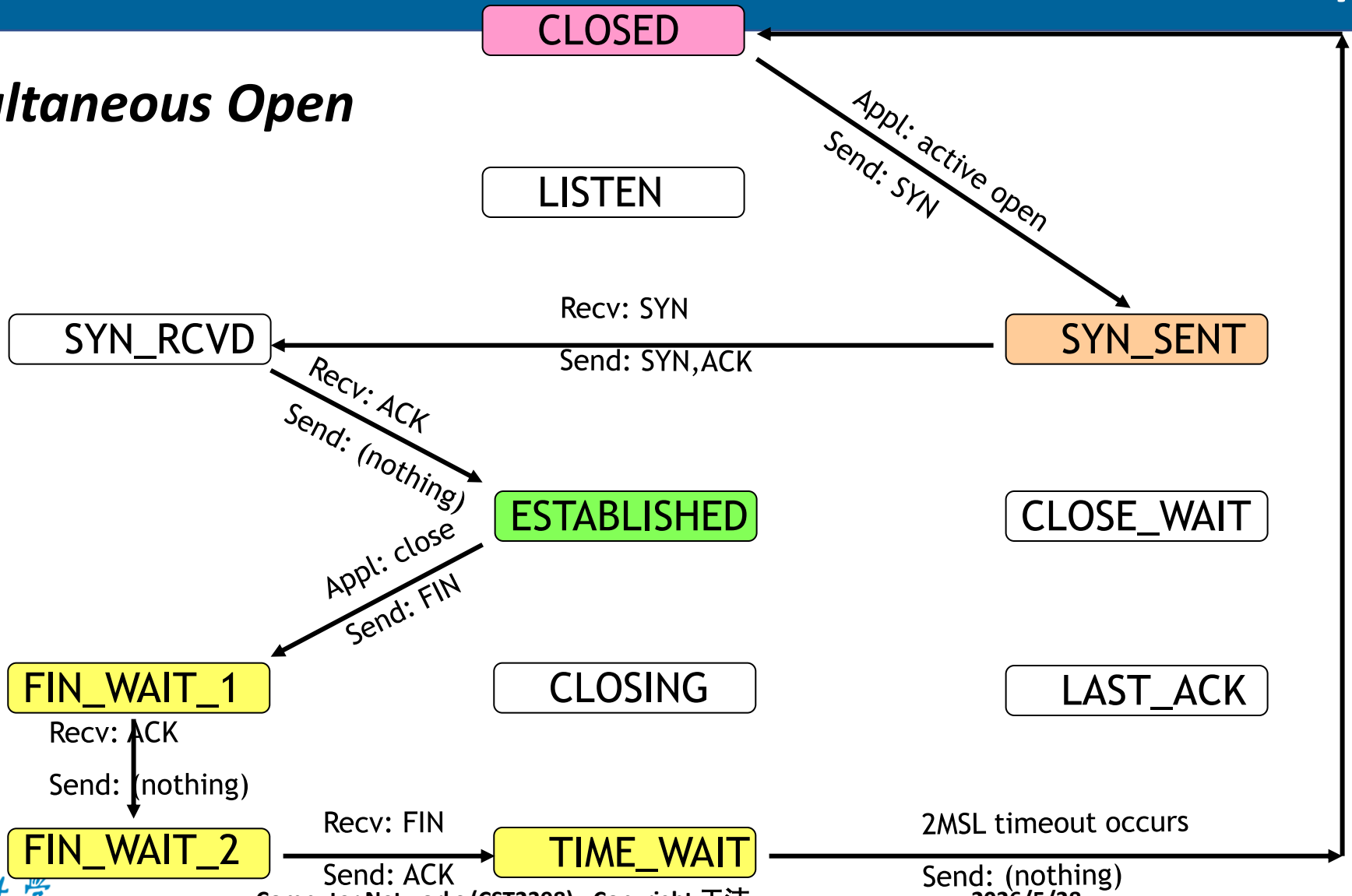
TCP Connection States - Normal

- ***Normal Client Open and Close***



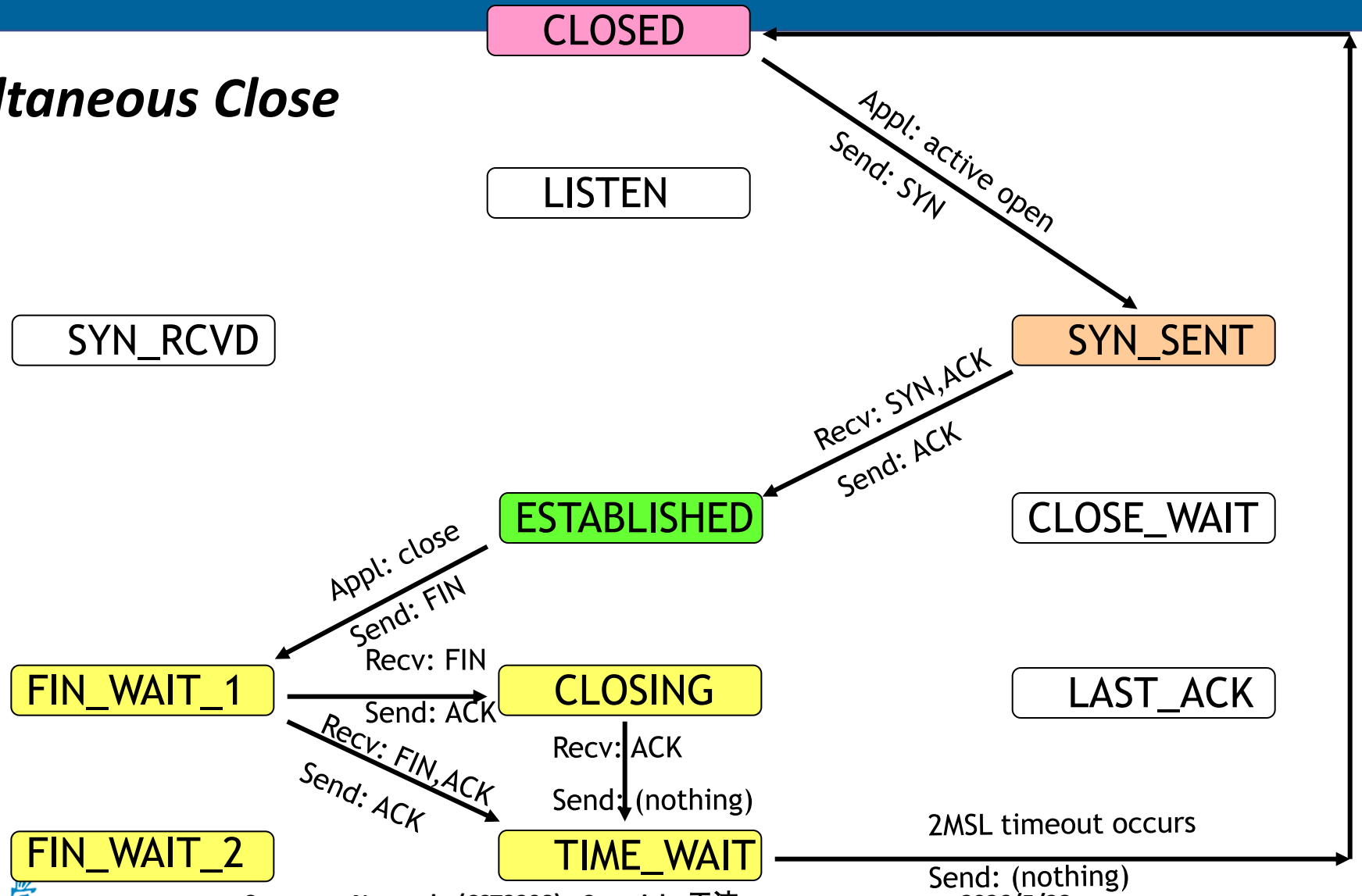
TCP Connection States – Simultaneous Open

- ***Simultaneous Open***



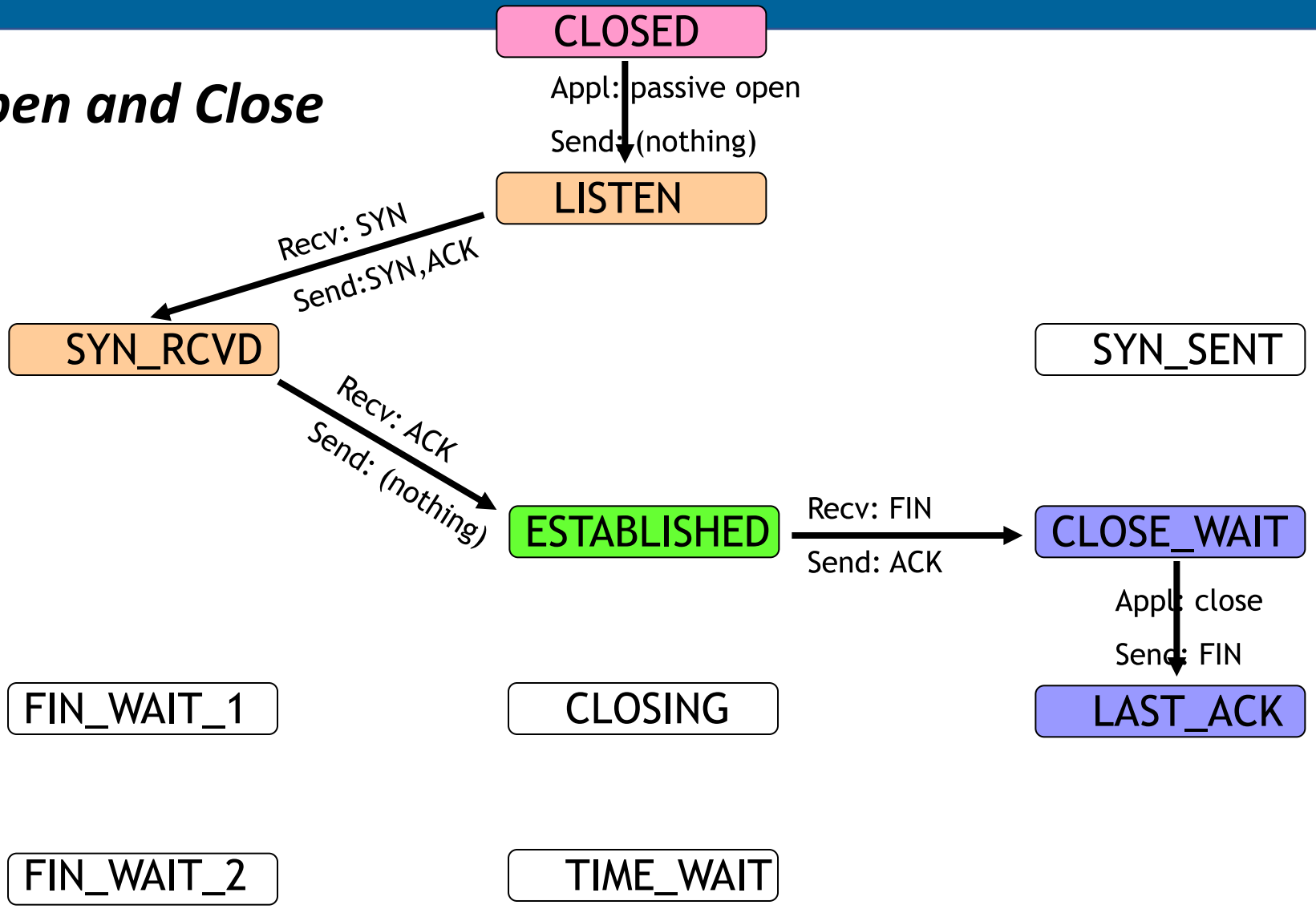
TCP Connection States – Simultaneous Close

- ***Simultaneous Close***



TCP Connection States – Server Open & Close

- ***Server Open and Close***



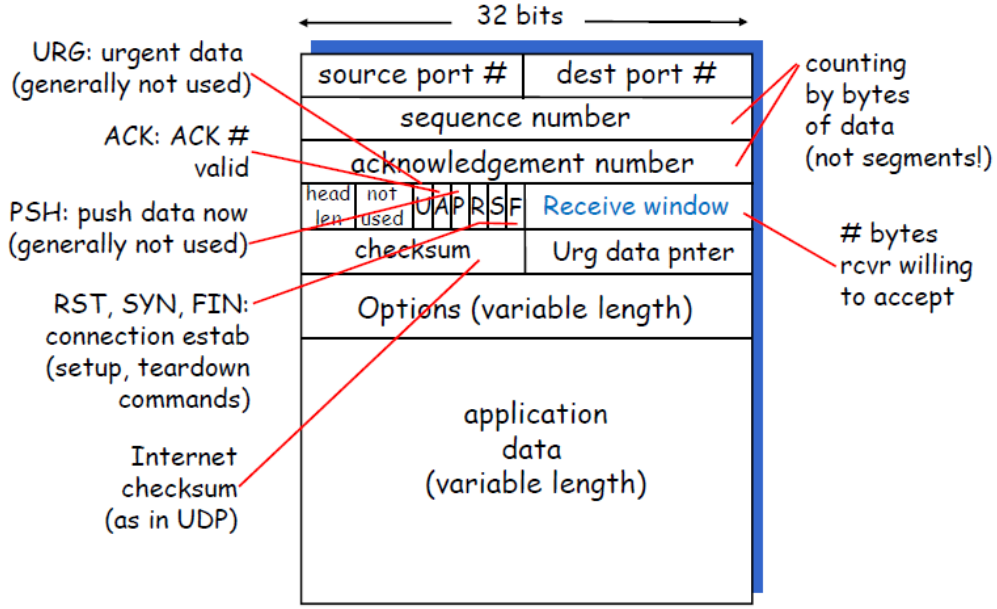
6.2 *Congestion Control*

TCP Features

- ***Connection-oriented, duplex, reliable byte-stream service with flow control.***
- ***Flow and Congestion Control: Sliding Window***
  - *TCP Reno, newReno: Go-Back-N*
  - *TCP SACK: Selective Repeat*
- ***32-bit ACK is cumulative:*** *ACK n -> all the n-1 bytes are received successfully.*
- ***TCP does not use NACK, but duplicated ACKs.***
- ***How does the sender know that byte n is lost?***
  - *Sender receives ACK n for multiple times -> lost or corrupted.*
  - *Timeout*

Flow/Congestion Control

- ***Two control mechanisms (control variable: window size)***
  - *Flow Control Mechanism (rwnd)*
    - *Physical meaning of rwnd: buffer size at the receiver*
    - *Receiver uses "Receive Window" field in the ACK to tell the sender rwnd*
  - *Congestion Control Mechanism (cwnd)*
    - *Complex, will see it on next page*
- ***Effective Window size = min(rwnd, cwnd)***



TCP Congestion Control (cwnd)

- ***Main Idea*: how to adjust *cwnd***
- ***Solution*: 2 phases**
  - ***Slow-start*** *(which is NOT slow): at the beginning*
    - *multiplicative increase of window size*

  - ***Congestion Avoidance*: *AIMD***
    - *additive increase of window size*

    - *multiplicative decrease of window size*

- ***Question*: How does TCP endpoint detect a congestion?**

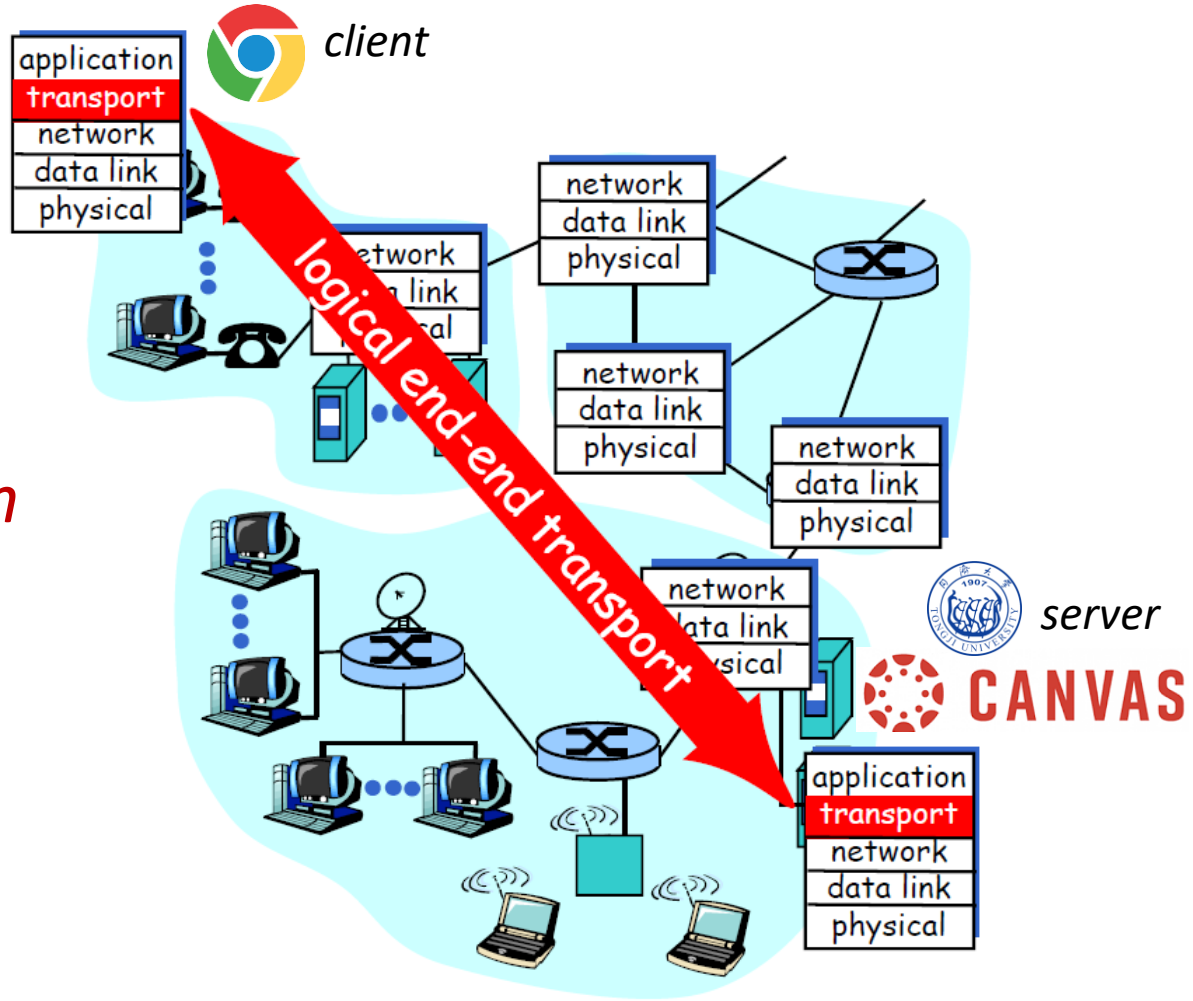
Why? To avoid congestions

No congestion

Possible congestion

Congestion and Packet Loss Detection

- ***When congestion happens:***
  - ***Routers*** *in the network: silently drop packets at the tail of the queue.*
  - ***End-systems*** *(host/server): packet loss*
- *End-system: packet loss ⇒ congestion*
- *New Question: How does an end system detect a packet loss?*
  - ***Retransmission Timeout (RTO)***
  - ***Duplicate ACKs****, usually triple (3*) ACKs*



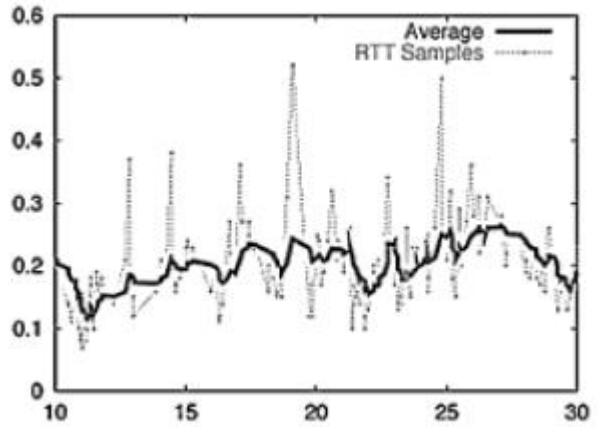
Detecting Packet Loss with RTO

- ***Retransmission TimeOut (RTO)***
  - *TCP sender sets a retransmission timer for a TCP segment.*
  - *No ACK for this segment is received -> Sender assumes this segment is lost.*
- ***RTO is dynamically updated.***
  - *The timeout period doubles for each timeout event.*
- ***RTO > RTT (which varies!)***
  - *Too short -> premature timeout -> unnecessary retransmission*
  - *Too long -> slow reaction to segment losses*

Estimating RTT

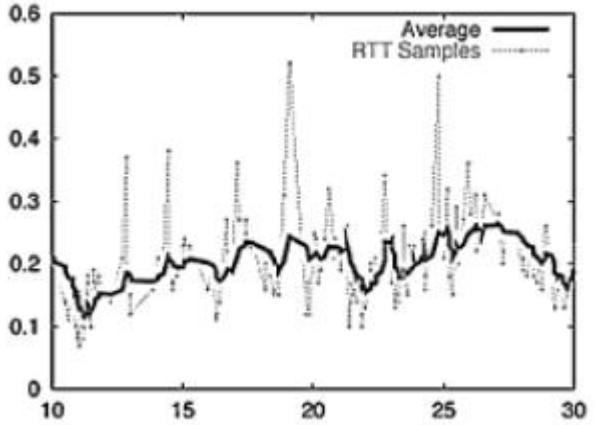
- ***TCP senders measures the SampleRTT, and estimate the EsimatedRTT***
  - **SampleRTT**: *the time from a segment transmission until receiving the ACK*
    - *Ignoring retransmissions, or cumulatively ACKed segments*
  - *It varies (sometimes a lot). We want a "smoother" RTT.*
- ***Solution: Exponential Weighted Moving Average (EWMA)***
  - *Influence of a given sample decreases exponentially with time. Typical x = 0.125.*

```
EstimatedRTT (new) =
      (1-x) (EstimatedRTT) + x (SampleRTT)
```



Setting the RTO

- ***RTO = EstimatedRTT + "safety margin"***
- ***The larger the variation in EstimatedRTT, the larger the safety margin.***
- ***Deviation is also weighted over time.***
  - *The influence of the deviation decreases exponentially with time.*



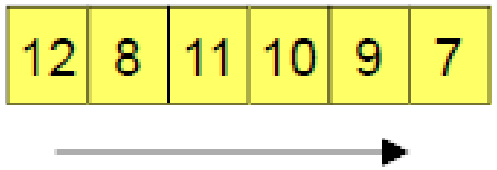
$$Timeout = EstimatedRTT + 4(Deviation)$$

$$Deviation = (1-x)(Deviation) + x\,|SampleRTT - EstimatedRTT|$$

Typical x = 0. 25 for Deviation

Detecting Packet Loss with Duplicated ACKs

- ***Receiver may send duplicated ACKs when a segment is lost.***
- ***So sender assumes a packet loss when receiving 3 duplicated ACKs.***
- ***But duplicated ACKs may also be generated***
  - *When there are out-of-order segment delivery, e.g.*



3 dupacks are also generated if
a packet is delivered at least 3 places
beyond its in-sequence location

Same
ACK #

  - *Fast Retransmit Mechanism: useful only if lower layers deliver segments "almost ordered", otherwise unnecessary.*

TCP Congestion Control Algorithm

- ***Two phases***
  - *Slow Start*
  - *Congestion Control/Avoidance*
- ***Two control variables***
  - *cwnd*
  - *ssthresh (slow start threshold)*
- ***Key idea: "probing" for the usable bandwidth/speed***
  - *Ideally: transmit as fast as possible -> set cwnd as large as possible*
  - *No loss (congestion): increase cwnd (until loss)*
  - *Loss happens: decrease cwnd, then "probe" again*

Slow Start (is not slow)

- ***Connection just established, set cwnd = 1 MSS***
  - *E.g. MSS = 500 bytes, RTT = 200 ms -> initial rate 20 kbps*
  - *At this point, the available bandwidth may be >> MSS/RTT.*
  - *So we want the rate to ramp up (increase) quickly.*

What is this?

- ***Action: Increase window size by 1 MSS for each new ACK received.***
- ***So cwnd grows exponentially with time in slow start phase.***
  - *Factor of 1.5 per RTT if every other segment is ACK'ed.*
  - *Factor of 2 per RTT if every segment is ACK'ed.*

The name does not tell its nature!

*Two mechanisms to reduce the amount of ACKs - the **Nagle** algorithm and **Delayed ACKs** - both described in RFC 1122.*

- ***Slow Start ends when window size reaches/exceeds ssthresh***

What is this?

TCP Slow Start Algorithm and Example

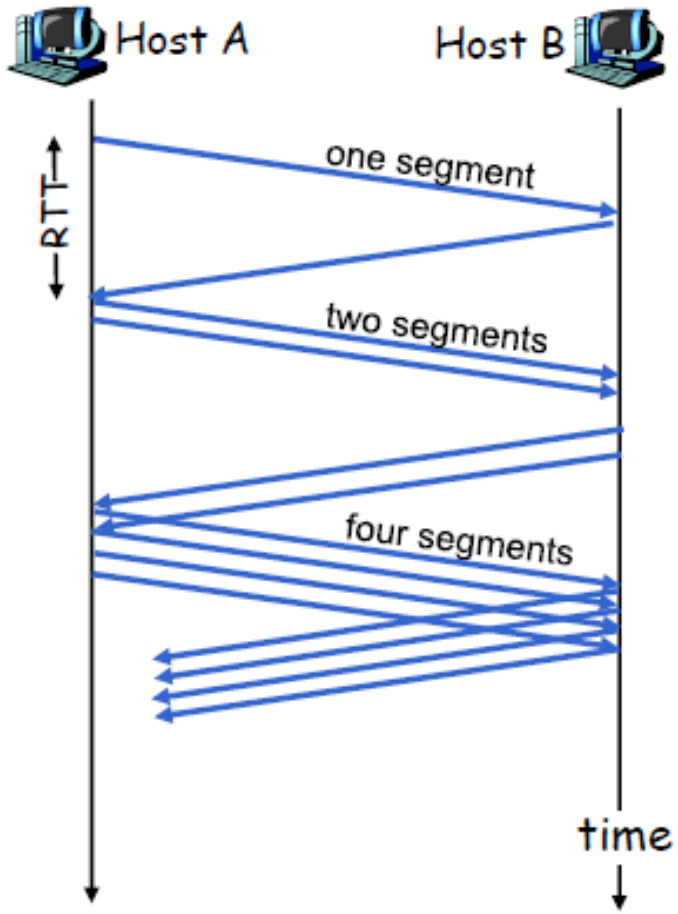
- ***Slow Start Algorithm (unit: MSS)***

Slowstart algorithm

initialize: Cwnd = 1
for (each segment ACKed)
    Cwnd++
until (loss event OR
    Cwnd > ssthresh)

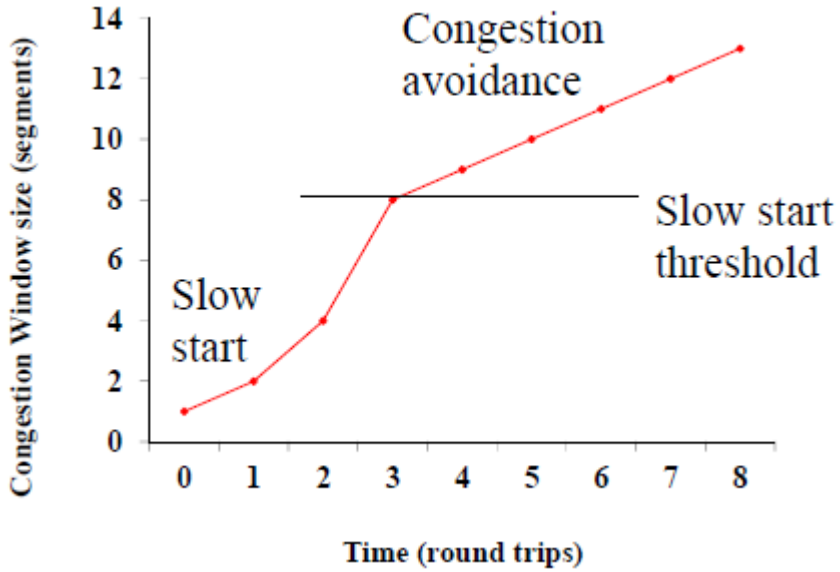
- *Exponential increase (per RTT) in window size.*
- *Loss event: timeout or three duplicated ACKs.*

- ***Objective: To ramp up cwnd quickly***



TCP Congestion Avoidance (1/3)

- ***TCP enters Congestion Avoidance phase when ssthresh is reached.***
- ***Action: Increase cwnd by 1/cwnd segment on each new ACK.***



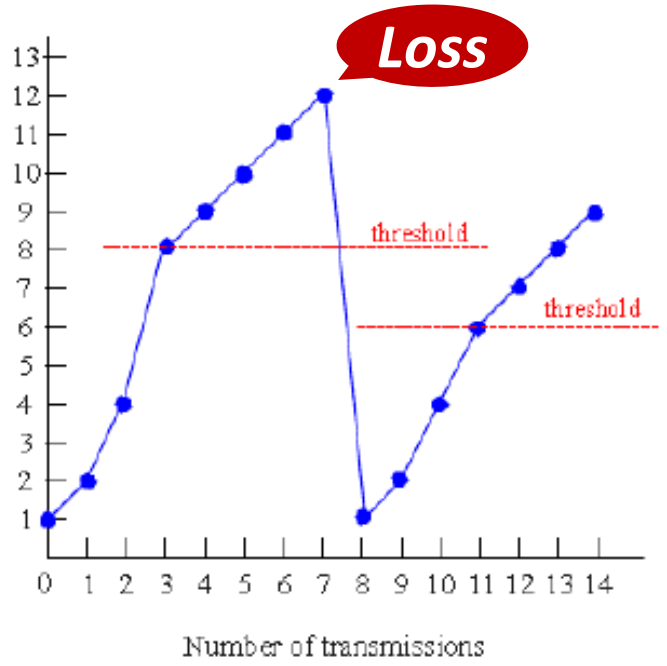
(Assume no loss, no timeout)

- ***Linear increase of cwnd in the congestion avoidance phase.***

TCP Congestion Avoidance (2/3)

- ***Congestion Avoidance Algorithm – TCP Tahoe (unit: MSS)***

```
Congestion avoidance
/* slowstart is over     */
/* Cwnd > ssthresh       */
Until (loss event) {
  every cwnd segments
ACKed:
      Cwnd++
  }
ssthresh = Cwnd/2
Cwnd = 1
perform slowstart
```



- ***Normal operation:***
  - *1/cwnd increase*
- ***Loss happens:***
  - *ssthresh = cwnd/2*
  - *cwnd = 1*

Too harsh?

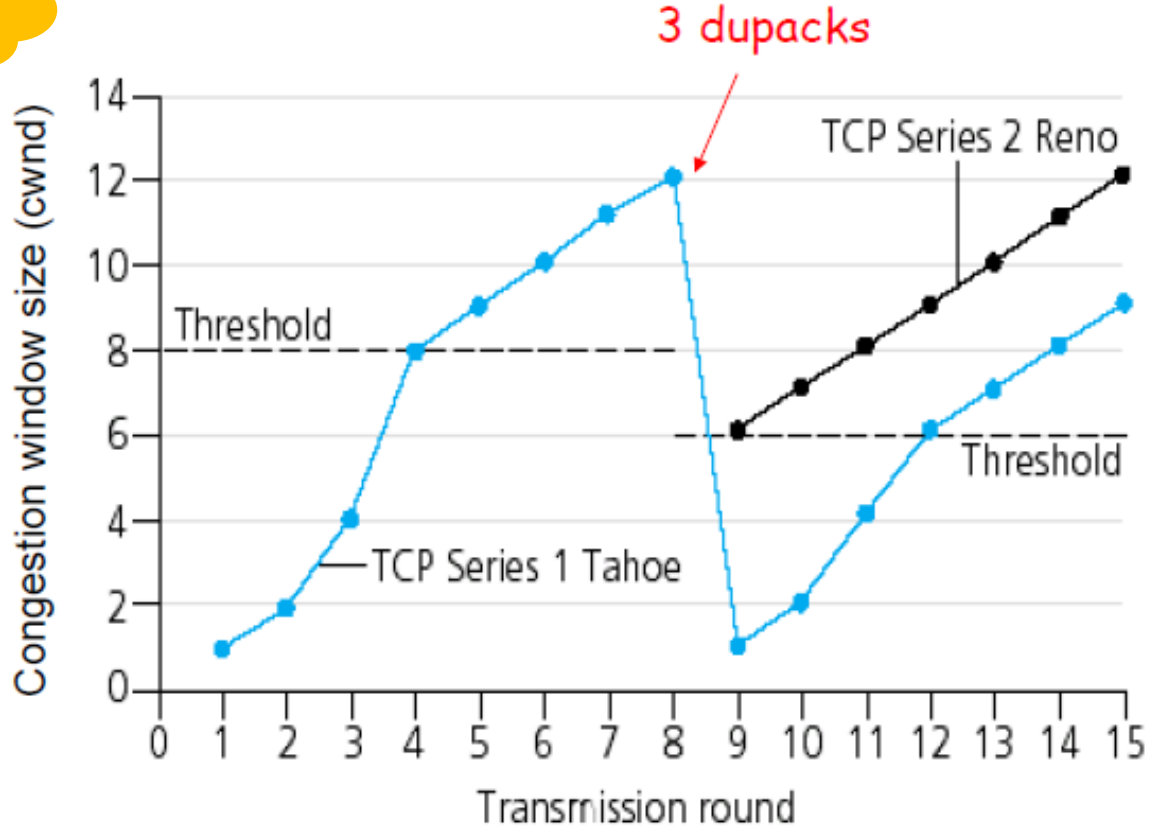
- ***When loss happens: congestion avoidance phase ends.***

TCP Congestion Avoidance (2/3)

- ***TCP-Reno: Timeout is more severe than duplicated ACKs.***
  - *After 3 duplicated ACKs*
    - *cwnd <- cwnd/2*
    - *Remains in Congestion Avoidance*
  - *After a timeout*
    - *cwnd = 1*
    - *Enters Slow Start*
- ***Tahoe v.s. Reno (no fast retransmit)***
  - *Same: timeout*
  - *Different: 3 duplicated ACKs*

AIMD

Fast Recovery



Fast Recovery (temporary phase/mode)

- ***When: After 3 duplicated ACKs***
- ***Why:*** *At least something got through – congestion is not so bad.*
  - *3 duplicated ACKs may be due to out-of-order delivery*
  - *Timer is still running, no need to be so harsh.*

Timeout is severer.

- ***Fast retransmit, then fast recovery – Read RFC 5681 Sec. 3.2***
  - *ssthresh = min (cwnd, rwnd)/2 (at least 2 MSS)*
  - ***Fast retransmit****: Retransmit the missing segment*
    - *cwnd = ssthresh + number of duplicated ACKs (3)*
  - ***Ends****: when a non-duplicated (new) ACK is received.*
    - *cwnd = ssthresh, enters congestion avoidance phase*

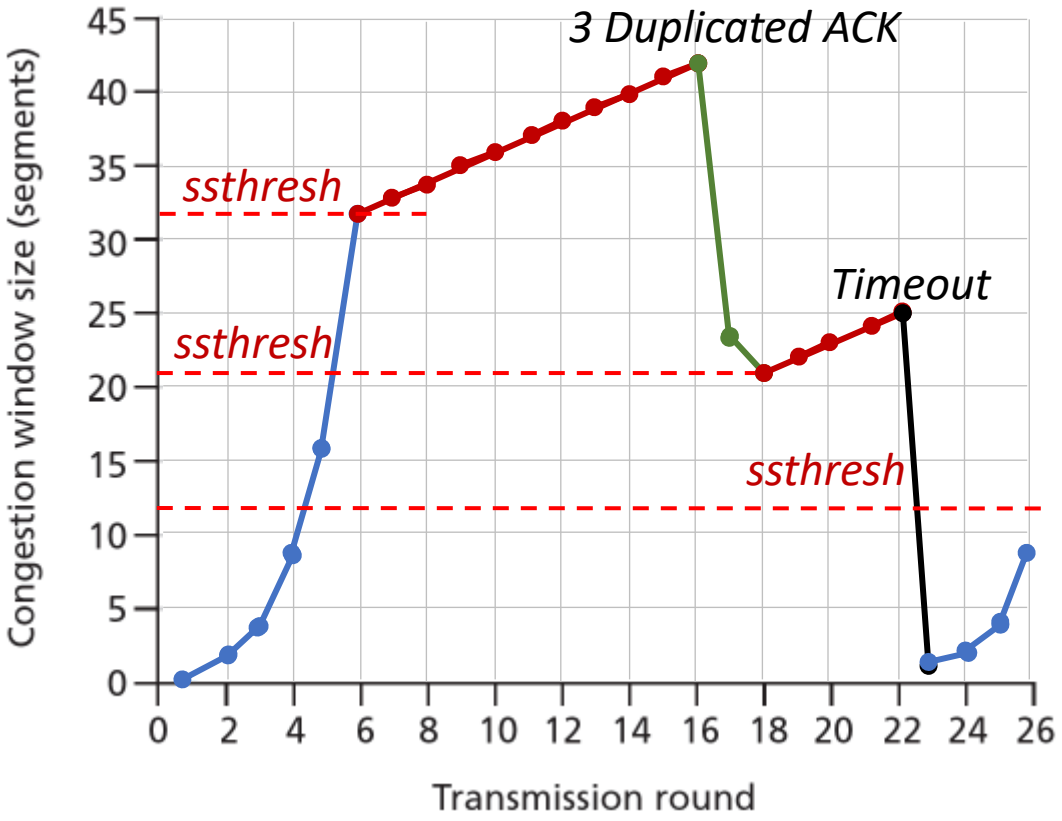
Temporary

Resetting to normal

Ex.6.1 TCP Congestion Control

Consider the following TCP congestion control process with TCP-Reno (w/ fast retransmit)

cwnd



1) *When is **TCP Slow-Start** operating?*
2) *When is **TCP Congestion Avoidance**?*
3) *What happened at round 16?*
4) *What happened at round 22?*
5) *What is the initial value of ssthresh at round 1, 18, and 24, respectively?*
6) *During which round is the 70th segment transmitted?*
7) *Suppose **TCP Tahoe** is used, and the same sequence of events happens, draw the curve.*

Summary: TCP Congestion Control (1/2)

- ***Start of Connection:*** *ssthresh is initialized.*
- ***Slow Start:*** *cwnd is increased by 1 with non-duplicated ACK.*
  - *cwnd increases by cwnd over one RTT. (If every segment is ACKed)*
  - *cwnd doubles every RTT. -> Exponential growth -> Not slow!*
- ***Congestion Avoidance:*** *after cwnd ≥ ssthresh, cwnd is increased by 1/cwnd with each non-duplicated ACK.*
  - *cwnd increases by 1 over one RTT.*

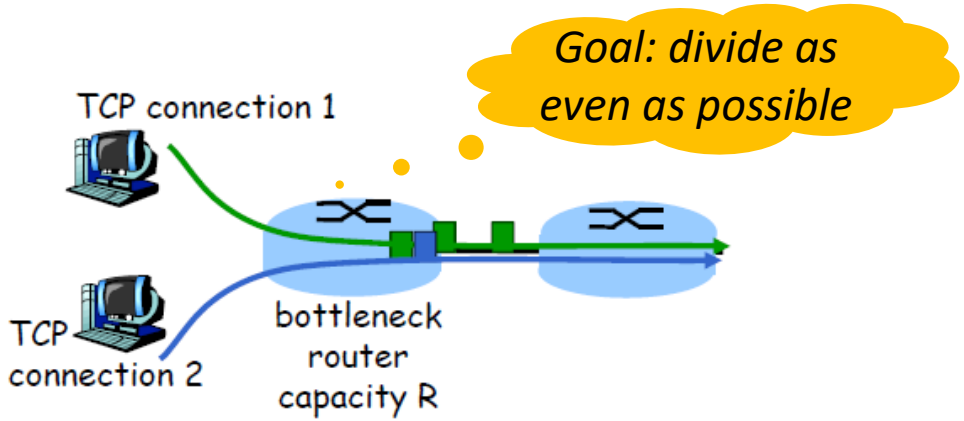
Summary: TCP Congestion Control (2/2)

- ***Fast Recovery: when receiving 3-duplicated ACKs, do the following***
  - *ssthresh is cut in half (ssthresh = ssthresh/2). Note old ssthresh = cwnd.*
  - *cwnd = ssthresh + 3*
  - *cwnd increases by 1 for every duplicated ACK.*
  - *When the retransmitted segment is finally ACKed, cwnd = ssthresh, and enters Congestion Avoidance Phase.*
- ***Retransmission Timeout: enters Slow Start***
  - *ssthresh = cwnd/2*
  - *cwnd = 1*

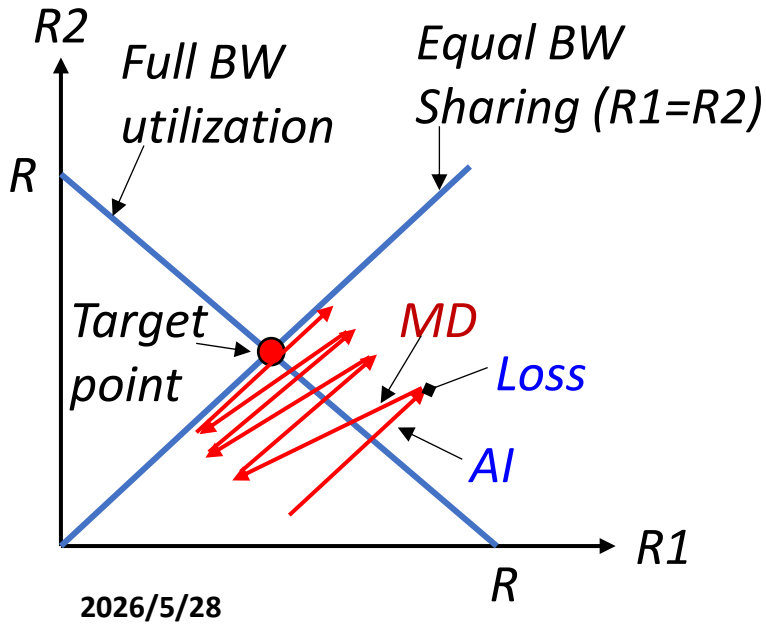
6.3 *QoS and Fairness*

TCP Performances Analysis - Fairness

- ***Ideal case*: Evenly divide the bandwidth.**
  - *If N TCP connections share the same bottleneck link, each connection should get 1/N of the link capacity.*



- ***TCP Fairness Analysis – 2 connections***
  - *R: the bottleneck link BW*
  - *R1, R2: throughput of each connection*
  - *Roughly: throughput ∝ cwnd*



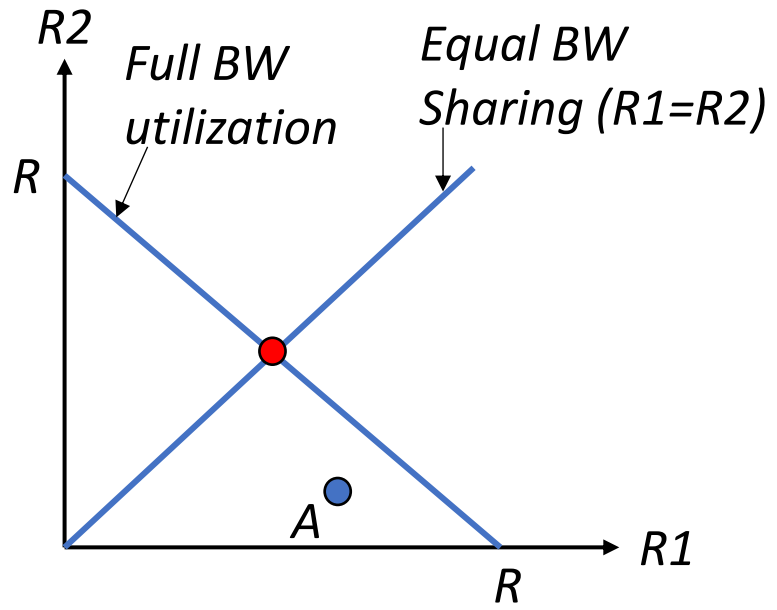
- ***AIMD*: move toward the target point**

Ex.6.2 TCP Fairness

Consider TCP connections share a bottleneck link of BW R.

- *Connection 1: TCP with AIMD*
- *Connection 2: TCP with modification*
  - *When a new ACK is received, increase cwnd by 1 (AI, as usual)*
  - *Upon congestion (3 duplicated ACKs), do nothing (maintains the same cwnd)*
- *Starting point: A*

What are the long-term throughput of the two connections? Is it fair?



Any other way to trick the system?

TCP Fairness: More Discussion

- ***Parallel (multiple) TCP Connections***
  - *One application can open parallel connections between the same pair of hosts.*
  - *Yes, the protocol allows this.*
  - *Actually, many web browsers do this.*
  - *E.g. a link of rate R is supporting 9 connections.*
    - *New app opens 1 TCP connection, getting rate R/10.*
    - *New app opens 11 TCP connections, getting rate 11R/20 > R/2!*

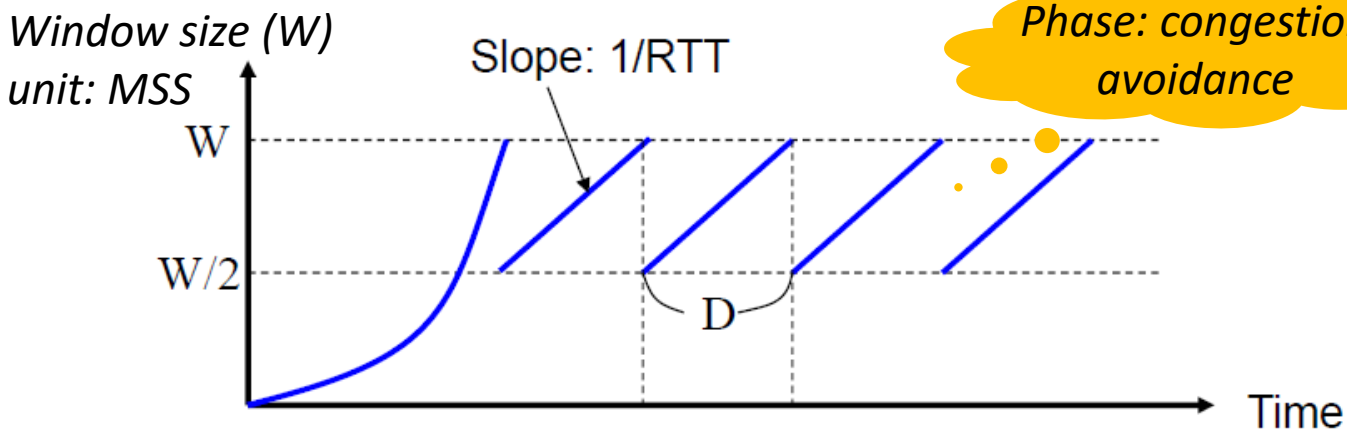
Is it fair?

- ***In reality, many multimedia apps use UDP instead of TCP, to avoid rate throttling by the congestion control mechanism in TCP.***

Good? Bad?

TCP Performances Analysis - Throughput

- ***Assumptions***
  - *TCP-Reno with Fast Recovery (ignore +3 -3)*
  - *No timeout*
  - *Infinite data, rwnd=∞*
  - *Single TCP connection, single router*
  - *System in steady-state (Congestion Avoidance)*
  - *Constant RTT*



- *When cwnd reaches W (max value before loss due to congestion), 3 duplicated ACKs are sent. Then cwnd=W/2.*
- *When loss happens, it takes* $D = W/2 \cdot RTT$ *to recover.*
- *In D seconds, the sender sends* $\frac{W}{2} + \left(\frac{W}{2} + 1\right) + \cdots + W = 3W^2/8$ *segments (MSS)*
- *Throughput* $R = \frac{3W^2/8}{D}$ *MSS/s, Loss rate* $L = \frac{1}{3W^2/8}$

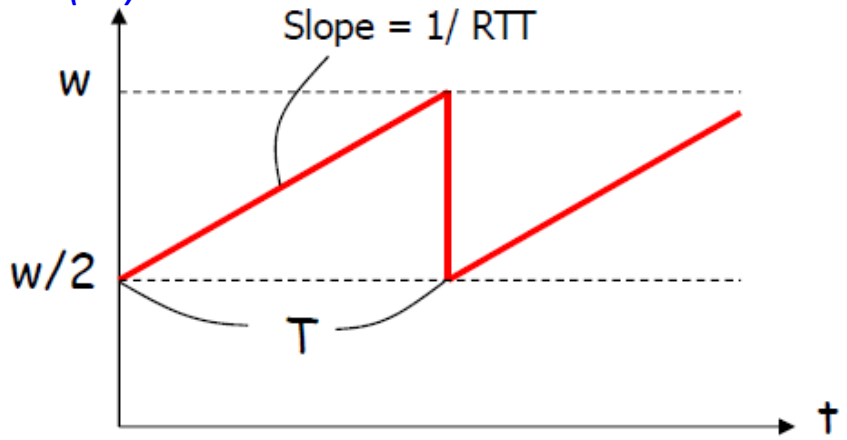
$$R \simeq \frac{1.25\ MSS}{RTT\ \sqrt{L}}$$

TCP over "Long, Fat Pipes"

- ***Long*, *fat* pipe -> networks with large *bandwidth-delay product***

RTT

Datarate (W)



- ✓ **Single TCP connection**
- ✓ Packet size = 1500 bytes
- ✓ RTT = 100ms
- ✓ Capacity (throughput) = 10Gbps

$$R \simeq \frac{1.25\ MSS}{RTT\ \sqrt{L}}$$

- ✓ **T ≥ 1.5 hrs!**
- ✓ **Packet drop rate ≤ 1 out of 5 billion packets!**

- ***Problem: poor utilization (It takes too long to fill the pipe again)***
  - *Or, to fill the pipe, packet drop rate must be smaller than the optics fiber PER.*
- ***Newer versions: TCP Vegas, BIC, Cubic, BBR, C2TCP…***

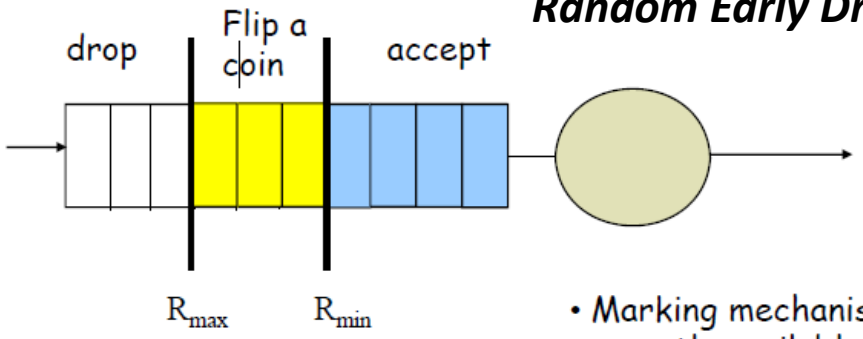
Newer TCP versions

Versions	Vegas	BIC	CUBIC	BBR
Time	1994 (Linux 2.26.20 Opt.)	INFOCOM 2004 (Linux 2.6.8-2.6.18)	SIGOPS 2008 (Linux 2.6.19, widely)	2016 (Linux 4.9+, Google)
Type	RTT-based	Loss-based	Loss-based	Measure-based
Design Logic	RTT-based control, drain buffers in the network	React to loss: Binary search to converge faster	React to loss: Cubic function to increase cwnd	Measure RTT and btn-BW to avoid congestion
Key Mechanisms	Adjust cwnd to drain buffers	Remember W_max, binary search	cwnd is a cubic function over time	4 stages to adjust cwnd
The good	Low delay, steady throughput	Converge quickly in high BDP networks	RTT fair, adapt quickly to wireless networks	High BW utilization, low delay, no bufferbloat
The bad	Easy to be pushed out when co-exist	Unfair to large RTT when co-exist	Bufferbloat, insensitive to BW changes	Unfair to other versions when co-exist

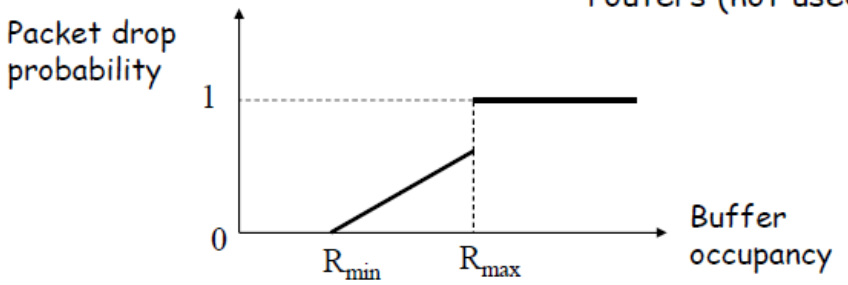
Recall AQM at Routers (last Chapter)

- ***Objective:*** *to minimize the delay (i.e. queue size) while giving necessary congestion information to the senders in time.*
- ***Simplest: Drop tail***
- ***Early Congestion Notification***
  - *Random Early Drop (RED)*
  - *Random Exponential Mask (REM)*
- ***Adaptive Virtual Queuing (AVQ)***
- ***Combinations/Hybrids***

Random Early Drop (RED)



• Marking mechanism that is currently available on Cisco routers (not used though!).



Possible Improvements for TCP (1/3)

- *Increase window size at a rate independent of RTT.*
  - *Eliminate bias in favor of connections with short RTTs.*
  - *Very difficult to choose a rate that works well over a wide range of RTTs.*
- *Base window adjustment on delays, not loss.*
  - *Problem: How to estimate the delays?*
  - *Compare RTT with the minimum RTT*
  - *Idea: (RTT – min RTT) measures queueing time in routers.*
  - *TCP-Vegas uses this protocol.*

Possible Improvements for TCP (2/3)

- *Explicit Congestion Notification:*
  - *Routers mark packets during congestion and/or send signal to the source.*
  - *Advantage: Sends source a signal to slow down without forced retransmission.*
  - *Disadvantage:*
    - *May bias in favor of shorter connections.*
    - *Routers have to implement marking*
- *Base decision to drop packets on # of packets of same connection in router, not simply on total # of packets.*
  - *Keeping track of all connections at router. More complex to implement.*

Possible Improvements for TCP (3/3)

- *Predict congestion based on current load and mark packets so that receiver can send feedback to the source:*
  - *Requires more sophisticated routers.*
- *Cheat and modify TCP window control.*
  - *Walrand & Varaiya say this is done all the time in commercial products.*

- *Problem with wireless links.*
  - *Not all packet losses are due to congestion.*
- *Problem with large bandwidth-delay product links*

So many modifications on TCP, why not so many on IP?

The long, fat pipe

6.4
TCP Sockets

However

- ***TCP/IP standard does NOT***
  - *Specify how to write distributed applications*
  - *Provide a way to start a process when a message arrives*
- ***The rendezvous problem***
  - *A program must be in the "waiting" state to establish communication before any requests arrive.*

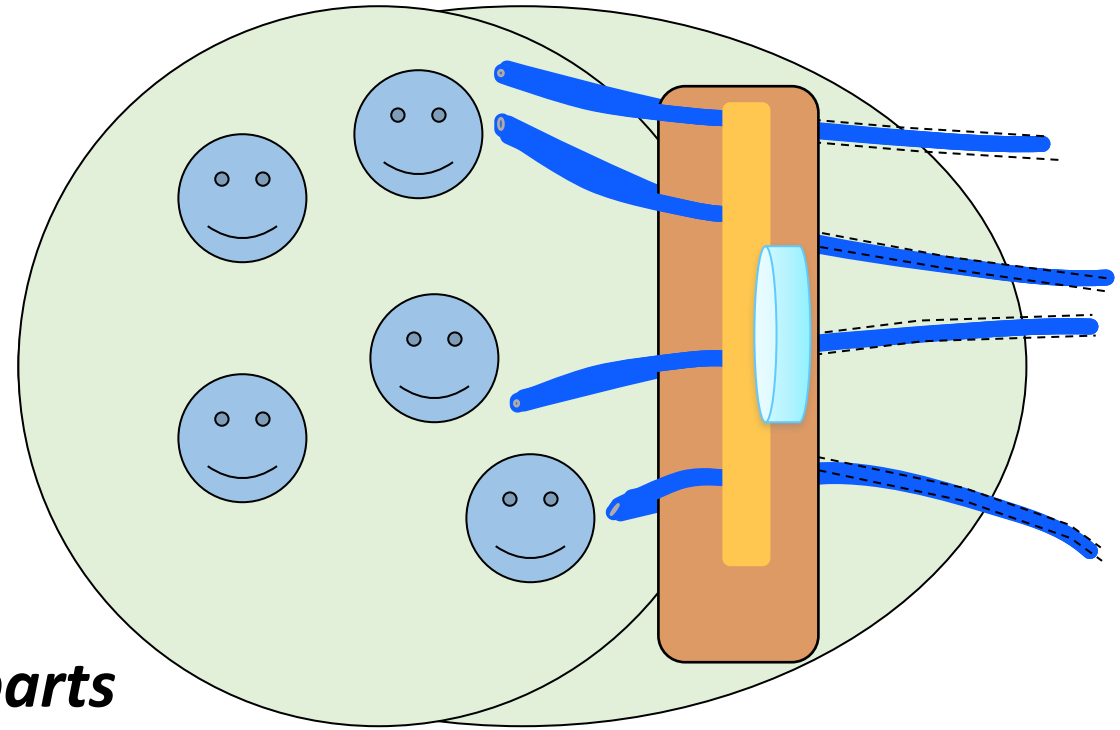
Overview

- ***Sockets** are the standard OS-level API for network communication since 1983.*
- *And likely will still be the core building block for all TCP-based systems, e.g.,*
  - *Web Server*
  - *Microservices*
- ***Interface between applications and Transport.***

- ***Key Uses***
  - *Custom protocol implementation*
    - *Finer-grained control*
  - *High-performance services*
    - *Direct use in C/C++ for min. overhead*
  - *Underlying every higher level API*
    - *Python socket, Node.js.net, HTTP/WebSocket…*
  - *Embedded and IoT Systems*
  - *Basis for proxies and firewalls*

Why do we need sockets 套接字

- ***We need to programmatically***
  - *Allocate resources for communication*
  - *Specify communication endpoints*
  - *Initiate a connection (client) or wait for incoming connections (server)*
  - *Send or receive data*
  - *Terminate a connection gracefully*
  - *Handle error conditions, etc.*
- ***So Transport software is built in two parts***
  - *Host-specific part: multiplex, global context*
  - *Application-specific part: maintains flow state, provides access point for higher layers*



However

- ***TCP/IP standard does NOT***
  - *Specify how to write distributed applications*
  - *Provide a way to start a process when a message arrives*
- ***The rendezvous problem***
  - *A program must be in the "waiting" state to establish communication before any requests arrive.*

Socket Abstraction

- ***Provides an endpoint for communication***
  - *Also provides buffering*
  - *Sockets are not bound to specific addresses at the time of creation*
- ***Identified by a small integer, the socket descriptor***
  - *Handle : program refers to it by this number (but programmer need not care)*
- ***Flexibility***
  - *Basic functions can be provided many different transport protocols, others specific to transport*
  - *Programmer specifies by type of service*
  - *A generic address structure*

Socket Programming

- ***Build client/server application that communicate using sockets***

Socket API

- *introduced in BSD4.1 UNIX, 1981*
- *explicitly created, used, released by apps*
- *client/server paradigm*
- *two types of transport service via socket API:*
  - *UDP (connectionless)*
  - *TCP (connection-oriented)*
- *Each socket has five components: protocol, local/dest IP, local/dest port*

"Create socket", "close socket"

"listen from any", "send to specific"

"blocking read", "non-blocking check", "timed block read"

"send to destination", "send to connected socket"

"remember (bind) local address", "remember farside address", "connect to farside address"

Clients vs. Servers (1/2)

- ***Who takes the first step?***
  - *Clients initiate peer-to-peer communications → active open*
  - *Servers wait for incoming communication requests → passive open*
- ***Lifetimes***
  - *Servers start execution before interaction begins, and continue to accept and process requests "forever"*
  - *Clients are short-lived*
- ***Ports***
  - *Servers wait for requests at a well-known port reserved for the service offered*
  - *Clients use arbitrary, non-reserved ports for communication*

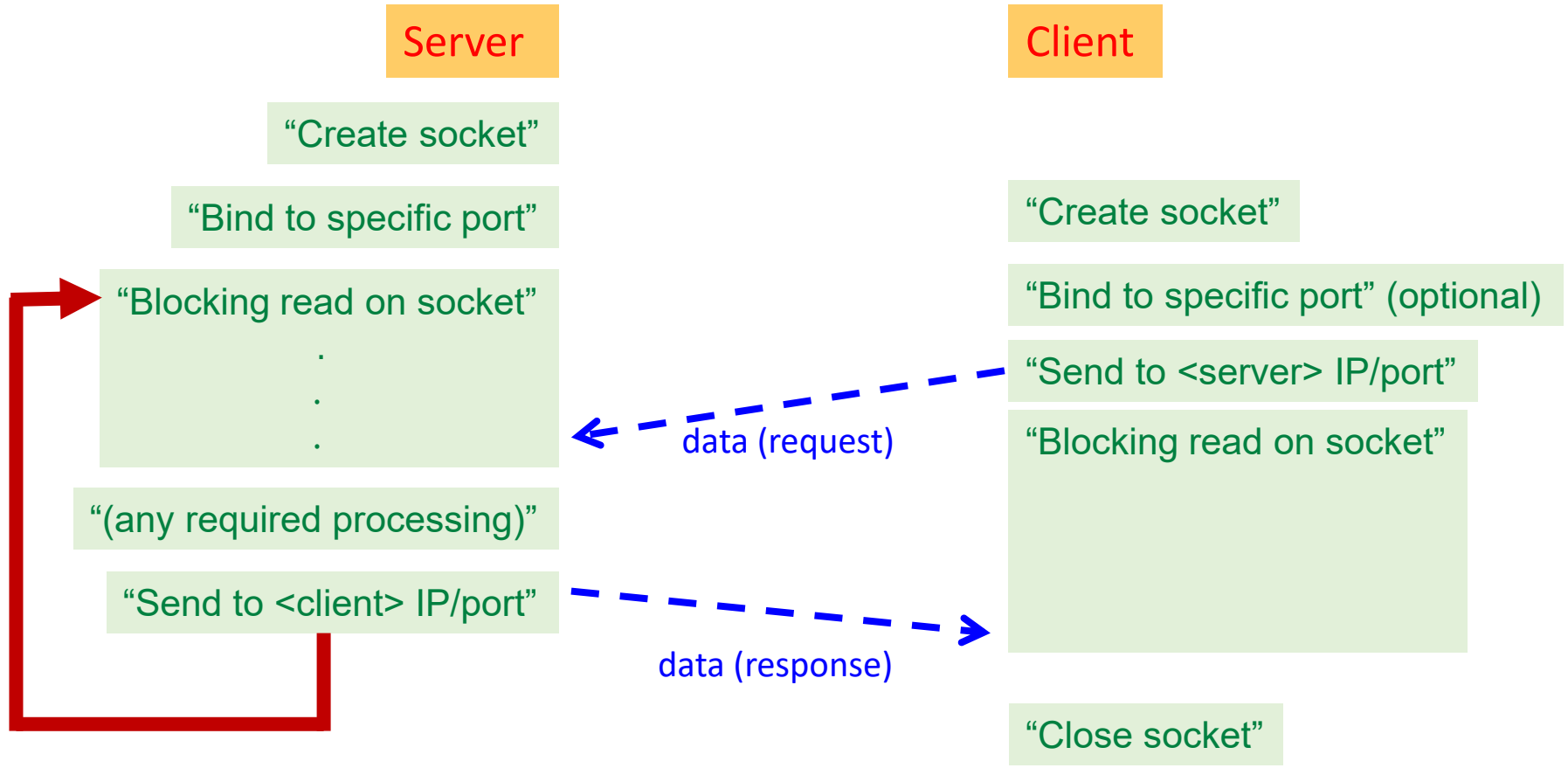
Clients vs. Servers (2/2)

- ***Complexity of clients***
  - *no special privileges required (usually)*
  - *overall, relatively easy to build*
- ***Complexity of servers***
  - *require special system privileges (usually)*
  - *may need to handle multiple requests concurrently*
  - *must protect themselves against all possible errors*
  - *generally more difficult to build*
- ***Both are usually implemented as application programs***

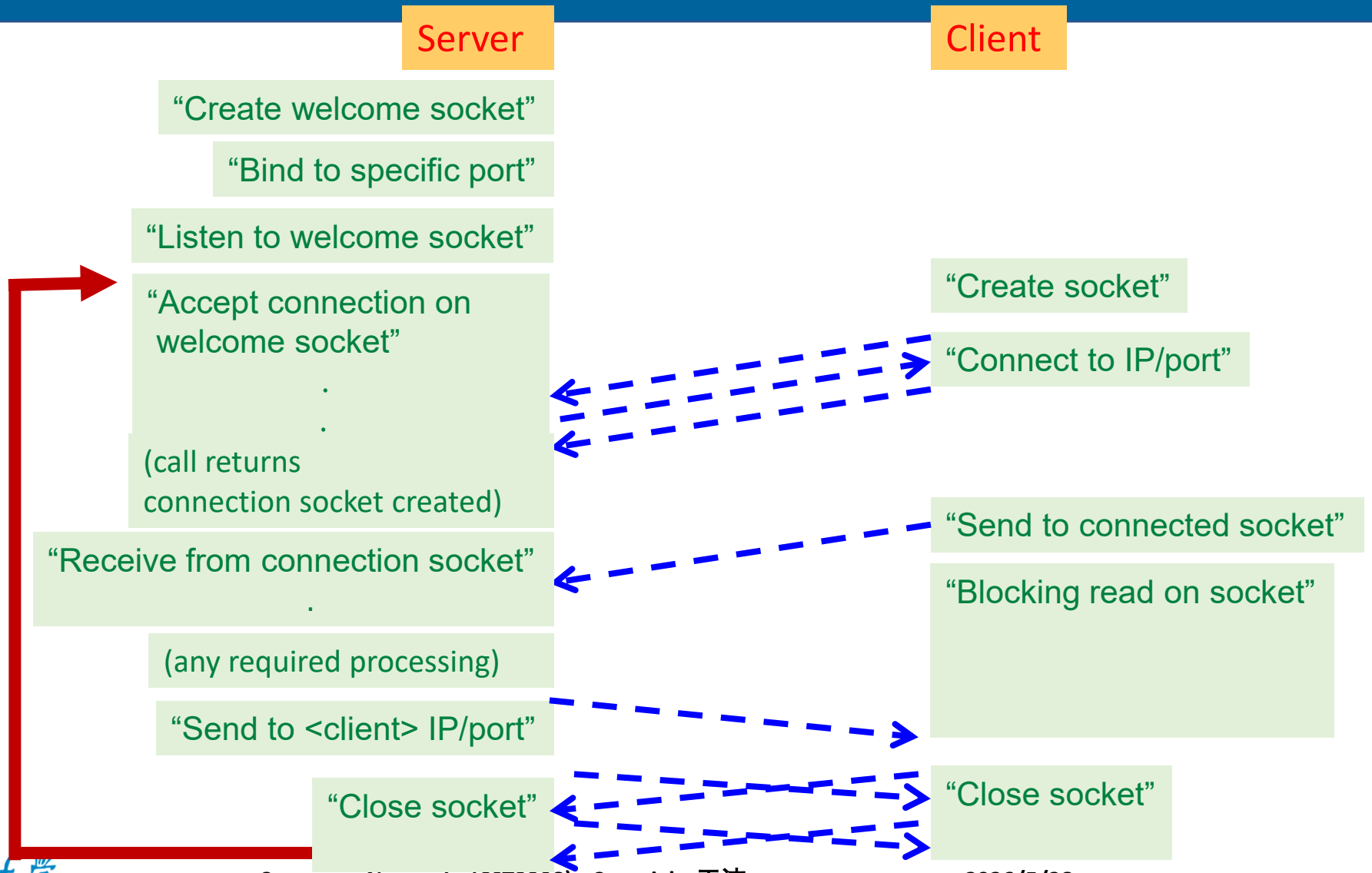
Types of Servers

	Connectionless	**Connection-Oriented**
Iterative (one-at-a-time)	Iterative Connectionless (*normal* UDP)	Iterative Connection-Oriented (*less common*)
Con-current	Concurrent Connectionless (*uncommon*)	Concurrent Connection-Oriented (*normal* TCP)

UDP Sockets – Logical Interaction



TCP Sockets – Logical Interaction



Evolution & Future Outlook

- ***From raw socket()/bind()/listen() to higher-level abstractions:***
  - *Frameworks (Netty, Boost.Asio, gnet)*
  - *Language runtime integration (Go net package, Java NIO)*
- ***Event-driven and async models now standard***
- ***Future Trends***
  - *Growing Importance in Cloud-Native*
  - *SmartNIC Offloading – Moving processing to hardware, freeing CPU cores*
  - *Compatibility Layers – Keeping socket API but accelerating performance transparently.*

Summary

- ***Functionality of the Transport Layer***
  - *Reliability -> ACK and retransmission*
  - *Flow Control -> rwnd*
  - *Congestion Control -> cwnd*
- ***Transmission Control Protocol***
  - *Endpoints: IP + port*
  - *Connection: 3-way handshake, 4-way handshake*
  - *Congestion Control: phases, variables, flavors, performances*
    - *TCP-Tahoe v.s.TCP-Reno*
    - *Performance: fairness and throughput*